

Linux

Fundamentos e Terminal

Do Zero ao Dominio Completo

Um guia completo e pesado para quem quer dominar Linux, o terminal e os fundamentos de sistemas operacionais.

10 Volumes • ~300 Paginas • Exemplos Praticos • Exercicios

Sumario

- Vol 1** Fundamentos do Sistema
- Vol 2** Terminal — Comandos Essenciais
- Vol 3** Busca e Manipulacao de Texto
- Vol 4** Pipes e Redirecionamentos
- Vol 5** Gerenciamento de Processos
- Vol 6** Redes no Linux
- Vol 7** Gerenciamento de Pacotes
- Vol 8** Bash Scripting
- Vol 9** Ferramentas para Desenvolvimento
- Vol 10** Linux para quem quer dominar C

Linux

Fundamentos e Terminal

Do Zero ao Domínio Completo

Um guia completo e pesado para quem quer dominar Linux, o terminal e os fundamentos de sistemas operacionais.

10 Volumes • ~300 Páginas • Exemplos Práticos • Exercícios

Sumário

- Vol 1** Fundamentos do Sistema
- Vol 2** Terminal — Comandos Essenciais
- Vol 3** Busca e Manipulação de Texto
- Vol 4** Pipes e Redirecionamentos
- Vol 5** Gerenciamento de Processos
- Vol 6** Redes no Linux
- Vol 7** Gerenciamento de Pacotes
- Vol 8** Bash Scripting
- Vol 9** Ferramentas para Desenvolvimento
- Vol 10** Linux para quem quer dominar C



Volume 1 — Fundamentos do Sistema

O que é Linux, como funciona por dentro, e por que você precisa entender isso antes de digitar qualquer comando.

1.1 O que é um Sistema Operacional?

Um **Sistema Operacional (SO)** é o software fundamental que gerencia os recursos de hardware de um computador e fornece serviços para os programas de aplicação. Ele atua como uma camada de abstração entre o hardware físico e o software do usuário.

Analogia

Pense no SO como o gerente de um restaurante: ele não cozinha (isso é o trabalho dos aplicativos), mas organiza a cozinha (hardware), distribui tarefas (CPU), gerencia o estoque (memória) e garante que os pedidos (processos) sejam atendidos na ordem correta.

Os principais componentes de um SO são:

- **Kernel (Núcleo):** O coração do sistema. Gerencia processos, memória, dispositivos e chamadas de sistema.
- **Shell:** Interface de linha de comando que interpreta comandos do usuário.
- **Sistema de Arquivos:** Organiza e armazena dados em dispositivos de armazenamento.
- **Gerenciador de Processos:** Controla a execução de programas.
- **Gerenciador de Memória:** Aloca e libera memória RAM para processos.
- **Drivers:** Permitem que o SO comunique-se com hardware específico.

```
graph TD
    subgraph "APLICAÇÕES DO USUÁRIO"
        A["(navegador, editor, compilador, etc.)"]
        B["BIBLIOTECAS (glibc, libstdc++, etc.)"]
    end
    subgraph "SHELL"
        C["(bash, zsh, fish)"]
    end
    subgraph "KERNEL"
        D["Gerenciador de Processos"]
        E["Gerenciador de Memória"]
        F["Sistema de Arquivos"]
        G["Drivers de Dispositivos"]
        H["Rede"]
    end
    subgraph "HARDWARE"
        I["(CPU, RAM, disco, placa de rede, etc.)"]
    end
    A --- B
    B --- C
    C --- D
    C --- E
    C --- F
    C --- G
    C --- H
    D --- I
    E --- I
    F --- I
    G --- I
    H --- I
```

1.2 O que é Linux?

Linux é um kernel de sistema operacional de código aberto, criado por **Linus Torvalds** em 1991. O termo "Linux" é frequentemente usado para se referir a sistemas operacionais completos baseados no kernel Linux, que incluem também ferramentas e bibliotecas do projeto **GNU**.

A história do Linux começou quando Linus, então estudante na Universidade de Helsinque, queria um sistema operacional similar ao UNIX para seu PC, mas não queria pagar pela licença do MINIX. Ele começou a escrever seu próprio kernel.

A Mensagem Original (1991)

"Hello everybody out there using minix — I'm doing a (free) operating system (just a hobby, won't be big and professional like gnu) for 386(486) AT clones."

— Linus Torvalds, comp.os.minix, 25 de agosto de 1991

Linux não é um sistema operacional completo por si só. É um **kernel**. O que chamamos de "Linux" no dia a dia (Ubuntu, Fedora, Debian) são na verdade **distribuições GNU/Linux** — combinações do kernel Linux com ferramentas do projeto GNU e outros softwares.

1.3 O Kernel Linux

O **kernel** é o componente central do sistema operacional. Ele tem acesso total ao hardware e gerencia todos os recursos do sistema. No Linux, o kernel é monolítico com módulos carregáveis — ou seja, a maior parte do código do kernel roda no mesmo espaço de memória, mas drivers e funcionalidades podem ser carregados dinamicamente.

1.3.1 Arquitetura do Kernel



1.3.2 Chamadas de Sistema (System Calls)

Chamadas de sistema são a interface entre o espaço do usuário e o kernel. Quando um programa precisa de um recurso privilegiado (acessar um arquivo, alocar memória, criar um processo), ele faz uma chamada de sistema. O kernel então executa a operação em modo privilegiado e retorna o resultado.

Exemplos de chamadas de sistema importantes:

Chamada	Descrição	Uso típico
<code>open()</code>	Abre um arquivo	<code>open("file.txt", O_RDONLY)</code>
<code>read()</code>	Lê dados de um arquivo	<code>read(fd, buffer, 1024)</code>
<code>write()</code>	Escreve dados em um arquivo	<code>write(fd, data, len)</code>
<code>fork()</code>	Cria um novo processo	<code>pid = fork()</code>
<code>execve()</code>	Executa um programa	<code>execve("/bin/ls", args, env)</code>
<code>mmap()</code>	Mapeia arquivo na memória	<code>mmap(NULL, size, prot, flags, fd, 0)</code>
<code>exit()</code>	Termina um processo	<code>exit(0)</code>

⚠ Importante para programadores C

Em C, você não chama as system calls diretamente. Você usa as **wrapper functions** da glibc (como `open()`, `read()`, `fork()`), que configuram os argumentos e executam a instrução `syscall` (ou `int 0x80` em x86, `syscall` em x86_64).

1.4 O Projeto GNU

O **Projeto GNU** foi iniciado por **Richard Stallman** em 1983 com o objetivo de criar um sistema operacional completamente livre, compatível com UNIX. GNU significa "GNU's Not Unix" — um acrônimo recursivo.

Até 1991, o projeto GNU tinha desenvolvido quase todos os componentes de um sistema operacional: compiladores (GCC), depuradores (GDB), bibliotecas (glibc), shell (bash), editores (Emacs), e utilitários. O que faltava era um kernel. Quando Linus Torvalds lançou o kernel Linux sob licença GPL, a combinação GNU + Linux formou um sistema operacional completo e funcional.

Por que "GNU/Linux" e não só "Linux"?

Richard Stallman e a Free Software Foundation argumentam que chamar o sistema apenas de "Linux" ignora a contribuição fundamental do projeto GNU. O kernel Linux sozinho não é um sistema operacional utilizável — são as ferramentas GNU que o tornam funcional.

1.4.1 Componentes GNU Essenciais

Componente	Descrição
GCC	GNU Compiler Collection — compilador C, C++, Fortran, etc.
GDB	GNU Debugger — depurador para C/C++
glibc	GNU C Library — biblioteca padrão C para Linux
Bash	Bourne Again Shell — shell padrão da maioria das distribuições
Coreutils	ls, cp, mv, rm, cat, etc. — utilitários básicos
Binutils	as, ld, objdump, nm — ferramentas para manipulação de binários
Make	Automação de compilação
Emacs	Editor de texto extensível

1.5 Shell e Terminal

É fundamental entender a diferença entre **terminal**, **console**, **shell** e **TTY** — termos frequentemente confundidos.

1.5.1 Terminal

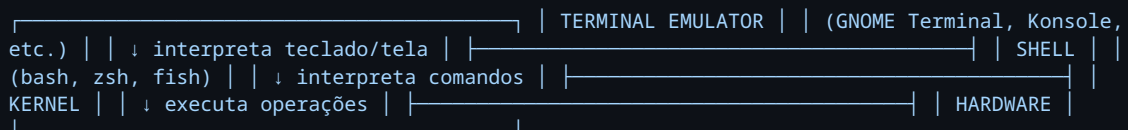
Um **terminal** (ou terminal emulador) é um programa que emula um dispositivo de terminal físico (como os antigos terminais VT100). Ele fornece uma janela onde você pode interagir com o shell. Exemplos: GNOME Terminal, Konsole, iTerm2, Windows Terminal, xterm.

1.5.2 Console

O **console** é o dispositivo físico de entrada/saída do sistema. Em servidores, é o monitor e teclado conectados diretamente à máquina. No Linux, os consoles virtuais são acessados com `Ctrl+Alt+F1` a `F6`. O ambiente gráfico geralmente roda no `F7` ou `F1` (dependendo da distribuição).

1.5.3 Shell

O **shell** é um programa interpretador de comandos. Ele lê o que você digita, interpreta e executa. O shell padrão no Linux é o **Bash** (Bourne Again Shell), mas existem alternativas como Zsh, Fish, Ksh, Dash.



1.5.4 TTY

TTY (Teletypewriter) é um conceito herdado dos mainframes. No Linux, um TTY é uma interface de terminal virtual. Cada sessão de login tem um TTY associado. Você pode ver o seu TTY com o comando `tty`:

```
$ tty
/dev/pts/0
```

Os TTYs físicos são `/dev/tty1` a `/dev/tty6`. Os TTYs pseudo (pts) são usados por terminais emulados em ambiente gráfico e conexões SSH.

Termo	O que é	Exemplo
Terminal	Emulador de terminal	GNOME Terminal, xterm
Console	Dispositivo físico de E/S	Monitor + teclado do servidor
Shell	Interpretador de comandos	Bash, Zsh
TTY	Interface de terminal virtual	<code>/dev/tty1</code> , <code>/dev/pts/0</code>
PTY	Pseudo-terminal (par master/slave)	<code>/dev/pts/0</code> (slave), <code>/dev/ptmx</code> (master)

1.6 Bash — O Shell Padrão

O **Bash** (Bourne Again Shell) é o shell padrão na maioria das distribuições Linux. É uma reimplementação do Bourne Shell (sh) com recursos adicionais do Korn Shell (ksh) e C Shell (csh).

1.6.1 Recursos do Bash

- **History:** Armazena comandos anteriores (arquivo `~/.bash_history`)

- **Tab completion:** Completa comandos, arquivos e variáveis com Tab
- **Aliases:** Atalhos para comandos longos
- **Variables:** Variáveis de ambiente e locais
- **Scripts:** Programação de automações
- **Job control:** Controle de processos em foreground/background
- **Redirection:** Redirecionamento de entrada/saída
- **Pipes:** Encadeamento de comandos

1.6.2 Arquivos de Configuração do Bash

Arquivo	Quando é executado	Propósito
<code>/etc/profile</code>	Login shell (todos usuários)	Configurações globais
<code>~/.bash_profile</code>	Login shell	Configurações pessoais
<code>~/.bashrc</code>	Shell interativo não-login	Aliases, funções, prompt
<code>~/.bash_logout</code>	Logout	Limpeza ao sair
<code>/etc/bash.bashrc</code>	Shell interativo (todos)	Configurações globais

🔍 Login Shell vs. Non-Login Shell

Um **login shell** é iniciado quando você faz login no sistema (SSH, console físico). Ele executa `/etc/profile` e `~/.bash_profile`. Um **non-login shell** é iniciado dentro de uma sessão já logada (abrir um terminal no ambiente gráfico). Ele executa `~/.bashrc`.

1.7 Distribuições Linux

Uma **distribuição Linux** (ou distro) é um sistema operacional completo construído a partir do kernel Linux, ferramentas GNU, e software adicional. Cada distribuição tem sua própria filosofia, sistema de pacotes, e público-alvo.

1.7.1 Principais Famílias

Família	Base	Gerenciador de Pacotes	Distros Populares
Debian	Debian	APT (dpkg)	Ubuntu, Linux Mint, Pop!_OS, Kali Linux
Red Hat	RHEL	RPM (DNF/YUM)	Fedora, CentOS, Rocky Linux, AlmaLinux
Arch	Arch	Pacman	Arch Linux, Manjaro, EndeavourOS
SUSE	SUSE	RPM (ZYpp)	openSUSE, SUSE Linux Enterprise
Gentoo	Gentoo	Portage	Gentoo, Calculate Linux

1.7.2 Como Escolher uma Distribuição

- **Iniciantes:** Ubuntu, Linux Mint, Pop!_OS — instalação fácil, grande comunidade
- **Desenvolvimento:** Fedora, Arch — software mais recente, ferramentas de desenvolvimento
- **Servidores:** Debian, Rocky Linux, Ubuntu Server — estabilidade, suporte LTS
- **Segurança/Cibersegurança:** Kali Linux, Parrot OS — ferramentas de pentest pré-instaladas
- **Minimalismo/Controle:** Arch, Gentoo — você constrói o sistema do zero

1.8 Boot e Inicialização

O processo de **boot** (inicialização) é a sequência de eventos que leva o computador do estado desligado até um sistema operacional funcional.

1.8.1 Fases do Boot

```
| 1. BIOS/UEFI | | ↓ POST (Power-On Self Test) |  
| ↓ Detecta hardware | | ↓ Localiza bootloader |  
| 2. Bootloader (GRUB2, systemd-boot) | | ↓ Carrega kernel e initramfs | | ↓ Passa parâmetros de  
boot | |  
| inicializa | | ↓ Detecta hardware, monta rootfs | | ↓ Executa processo init (PID 1) |  
| 3. Kernel | | ↓ Descompacta e  
| 4. Init System (systemd, OpenRC, runit) | | ↓  
Inicializa serviços | | ↓ Configura rede, logging, etc. | | ↓ Apresenta login |
```

1.8.2 BIOS vs. UEFI

Característica	BIOS (Legado)	UEFI (Moderno)
Idade	1981	2005
Partição de boot	MBR	GPT
Tamanho máximo do disco	2 TB	9.4 ZB (zettabytes)
Partições	4 primárias	128 partições
Secure Boot	Não	Sim
Interface	Texto	Gráfica
Bootloader	GRUB legado	GRUB2, systemd-boot

1.9 systemd — O Sistema de Inicialização

O **systemd** é o sistema de inicialização e gerenciamento de serviços padrão na maioria das distribuições Linux modernas (Ubuntu, Fedora, Debian, CentOS, Arch). Ele substituiu o antigo SysVinit.

1.9.1 Conceitos do systemd

- **Units:** Recursos que o systemd gerencia. Cada unit tem um arquivo de configuração.
- **Services:** Processos que o systemd inicia e monitora.
- **Targets:** Estados do sistema (equivalentes aos runlevels).
- **Timers:** Agendamento de tarefas (alternativa ao cron).
- **Mounts:** Pontos de montagem gerenciados pelo systemd.
- **Sockets:** Ativação de serviços sob demanda.

1.9.2 Comandos systemd Essenciais

```
# Verificar status de um serviço
$ systemctl status sshd

# Iniciar um serviço
$ sudo systemctl start nginx

# Parar um serviço
$ sudo systemctl stop nginx

# Habilitar serviço para iniciar no boot
$ sudo systemctl enable nginx

# Desabilitar serviço no boot
$ sudo systemctl disable nginx

# Recarregar configuração de um serviço
$ sudo systemctl reload nginx

# Reiniciar um serviço
$ sudo systemctl restart nginx

# Listar todos os serviços ativos
$ systemctl list-units --type=service --state=active

# Ver logs de um serviço
$ journalctl -u nginx

# Ver logs em tempo real
$ journalctl -f

# Ver logs desde o último boot
$ journalctl -b
```

1.9.3 Targets do systemd (Runlevels)

Target	Descrição	Runlevel Equivalente
<code>poweroff.target</code>	Desligamento	0
<code>rescue.target</code>	Modo de recuperação	1
<code>multi-user.target</code>	Multiusuário sem GUI	3
<code>graphical.target</code>	Multiusuário com GUI	5

```
# Mudar para modo de recuperação
$ sudo systemctl isolate rescue.target

# Ver target atual
$ systemctl get-default

# Mudar default para multiusuário (sem GUI)
$ sudo systemctl set-default multi-user.target
```

1.10 Estrutura do Sistema de Arquivos (FHS)

O **Filesystem Hierarchy Standard (FHS)** define a estrutura de diretórios em sistemas Linux. Conhecer essa estrutura é essencial para navegar e entender o sistema.

```
/ |— bin/ → Comandos essenciais (binários do sistema) |— boot/ → Arquivos de boot (kernel,
initramfs, GRUB) |— dev/ → Arquivos de dispositivos (hardware virtual) |— etc/ → Arquivos de
configuração do sistema |— home/ → Diretórios pessoais dos usuários |— lib/ → Bibliotecas
essenciais compartilhadas |— lib64/ → Bibliotecas de 64 bits |— media/ → Pontos de montagem
para mídias removíveis |— mnt/ → Pontos de montagem temporários |— opt/ → Software adicional
opcional |— proc/ → Sistema de arquivos virtual (processos) |— root/ → Diretório pessoal do
root |— run/ → Dados voláteis de runtime (PID files, sockets) |— sbin/ → Comandos essenciais de
administração |— srv/ → Dados de serviços (web, ftp, etc.) |— sys/ → Sistema de arquivos
virtual (sysfs) |— tmp/ → Arquivos temporários |— usr/ → Hierarquia secundária (programas do
usuário) |— bin/ → Comandos não-essenciais |— include/ → Headers de C/C++ |— lib/ →
Bibliotecas não-essenciais |— local/ → Software instalado localmente |— sbin/ → Comandos de
administração não-essenciais |— share/ → Dados compartilhados (man pages, docs) |— src/ →
Código-fonte do kernel |— var/ → Dados variáveis (logs, caches, spool) |— log/ → Logs do
sistema |— mail/ → Caixas de correio |— spool/ → Filas de impressão, email, cron |— tmp/
→ Arquivos temporários persistentes |— lost+found/ → Recuperação de arquivos corrompidos (ext4)
```

1.10.1 Diretórios Importantes para Desenvolvedores C

Diretório	Conteúdo	Relevância para C
<code>/usr/include</code>	Headers do sistema (<code>stdio.h</code> , <code>stdlib.h</code>)	Essencial para compilação
<code>/usr/lib</code>	Bibliotecas compartilhadas (<code>.so</code>)	Linkedição dinâmica
<code>/usr/local</code>	Software instalado manualmente	Instalação de libs próprias
<code>/proc</code>	Informações de processos	Debug, análise de memória
<code>/dev</code>	Dispositivos (disco, terminal, etc.)	Acesso a hardware
<code>/sys</code>	Informações do kernel	Hardware, drivers, kernel

1.11 Usuários, Root, UID e GID

Linux é um sistema **multiusuário**. Cada usuário tem uma identidade única e permissões específicas.

1.11.1 O Usuário Root

O **root** (UID 0) é o superusuário do sistema. Ele tem acesso irrestrito a todos os arquivos e pode executar qualquer operação. O poder do root é também seu maior perigo — um comando errado pode destruir o sistema.

Regra de Ouro

NUNCA use o root para tarefas do dia a dia. Use uma conta de usuário regular e eleve privilégios apenas quando necessário com `sudo`. Isso protege o sistema de erros acidentais e limita o impacto de malware.

1.11.2 UID e GID

Faixa de UID	Tipo de Usuário
0	root (superusuário)
1-999	Usuários de sistema (daemon, bin, www-data, etc.)
1000+	Usuários regulares

1.11.3 Arquivo /etc/passwd

Cada linha do `/etc/passwd` representa um usuário, com campos separados por `:`:

1.11.4 Arquivo /etc/shadow

As senhas criptografadas são armazenadas em `/etc/shadow`, que só é legível pelo root:

Os campos são: usuário, hash da senha, última mudança, dias mínimos, dias máximos, dias de aviso, dias de inatividade, data de expiração, reservado.

1.12 Permissões de Arquivos

Linux usa um modelo de permissões baseado em três classes de usuários: **proprietário (owner)**, **grupo (group)**, e **outros (others)**. Cada classe pode ter permissões de **leitura (r)**, **escrita (w)**, e **execução (x)**.

1.12.1 Notação Simbólica (rwx)

Permissão	Arquivo	Diretório
r (read)	Ler conteúdo	Listar arquivos
w (write)	Modificar conteúdo	Criar/deletar arquivos
x (execute)	Executar como programa	Acessar (cd) o diretório

```
$ ls -l arquivo.txt
-rwxr-xr-- 1 usuario grupo 1234 Jun 14 10:00 arquivo.txt
```

- permissões para outros (r--)
- permissões para o grupo (r-x)
- permissões para o proprietário (rwx)
- tipo de arquivo (- = arquivo regular)

1.12.2 Notação Numérica (Octal)

Cada permissão tem um valor numérico:

Permissão	Valor
r (read)	4
w (write)	2
x (execute)	1

Os valores são somados para cada classe:

```
rwX = 4+2+1 = 7
r-X = 4+0+1 = 5
r-- = 4+0+0 = 4

chmod 754 arquivo.txt → rwxr-xr--
```

1.12.3 chmod — Alterar Permissões

Notação simbólica

```
$ chmod u+x script.sh      # Adiciona execução para o dono
$ chmod g-w arquivo.txt    # Remove escrita do grupo
$ chmod o=r arquivo.txt    # Define leitura apenas para outros
$ chmod a+x programa       # Adiciona execução para todos

# Notação octal
$ chmod 755 script.sh      # rwxr-xr-x (scripts executáveis)
$ chmod 644 arquivo.txt    # rw-r--r-- (arquivos comuns)
$ chmod 700 chave.pem      # rwx----- (chaves privadas)
$ chmod 777 diretorio/     # rwxrwxrwx (EVITE!)
```

NUNCA use chmod 777

`chmod 777` dá permissão total a todos os usuários. Isso é uma falha de segurança grave. Use o princípio do **mínimo privilégio**: dê apenas as permissões estritamente necessárias.

1.12.4 chown — Alterar Proprietário

```
# Mudar apenas o dono
$ sudo chown root arquivo.txt

# Mudar dono e grupo
$ sudo chown usuario:grupo arquivo.txt

# Mudar recursivamente em um diretório
$ sudo chown -R usuario:grupo /var/www/

# Mudar apenas o grupo
$ sudo chown :grupo arquivo.txt
```

1.12.5 umask — Máscara de Permissões Padrão

O `umask` define as permissões padrão para novos arquivos e diretórios. É subtraído das permissões máximas.

```
$ umask
0002

# Permissões máximas: arquivo 666, diretório 777
# Arquivo novo: 666 - 002 = 664 (rw-rw-r--)
# Diretório novo: 777 - 002 = 775 (rwxrwxr-x)

# Alterar umask temporariamente
$ umask 077
# Agora: arquivo 600, diretório 700 (apenas o dono)
```

🔍 Permissões Especiais

Além de rwx, existem três permissões especiais:

- **SUID (4)**: Arquivo executa com privilégios do dono. Ex: `passwd` (rwsr-xr-x)
- **SGID (2)**: Arquivo executa com privilégios do grupo; diretório herda grupo. Ex: `/usr/bin/wall`
- **Sticky Bit (1)**: Apenas o dono pode deletar arquivos no diretório. Ex: `/tmp` (rwxrwxrwt)

```
$ chmod 4755 programa    # SUID
$ chmod 2755 diretorio   # SGID
$ chmod 1777 /tmp        # Sticky Bit
```

1.13 Permissões Avançadas — ACLs

As **Access Control Lists (ACLs)** permitem definir permissões mais granulares que o modelo tradicional `rxw`. Com ACLs, você pode dar permissões específicas para usuários e grupos individuais além do dono e grupo principal.

```
# Ver ACLs de um arquivo
$ getfacl arquivo.txt

# Definir ACL: dar permissão de leitura para um usuário específico
$ setfacl -m u:outro_usuario:r arquivo.txt

# Definir ACL para um grupo
$ setfacl -m m:grupo:rw arquivo.txt

# Remover todas as ACLs
$ setfacl -b arquivo.txt

# Definir ACL padrão para novos arquivos em um diretório
$ setfacl -d -m u:outro_usuario:rxw diretorio/
```

Quando usar ACLs?

Use ACLs quando precisar de controle de acesso mais fino do que o modelo tradicional permite. Por exemplo: um diretório de projeto onde vários usuários precisam de acesso, mas não fazem parte do mesmo grupo principal.

🔧 Exercícios — Volume 1

1. Execute `uname -a` e interprete cada campo da saída. Qual a versão do seu kernel?
2. Use `tty` e `whoami`. Qual seu TTY atual e UID? Use `id` para confirmar.
3. Explore `/proc`: execute `cat /proc/cpuinfo | grep "model name"` e `cat /proc/meminfo | head -5`.
4. Crie um arquivo `teste.txt`, defina permissões `640` e verifique com `ls -l`. O que significa cada permissão?
5. Crie um diretório `projeto`, mude o grupo para `users` com `chown`, e defina SGID com `chmod 2775`. Crie um arquivo dentro e verifique o grupo.
6. Use `systemctl list-units --type=service` e identifique 5 serviços essenciais do sistema.
7. Verifique o `umask` padrão. Altere para `077`, crie um arquivo e observe as permissões. Volte ao padrão original.
8. Instale `acl` e use `setfacl` para dar permissão de leitura em um arquivo para outro usuário do

sistema.



Volume 2 — Terminal: Comandos Essenciais

Os comandos que você usará todos os dias. Cada comando com dezenas de exemplos, flags e casos de uso reais.

2.1 pwd — Print Working Directory

O comando `pwd` exibe o caminho absoluto do diretório atual. Simples, mas fundamental para saber onde você está no sistema de arquivos.

```
$ pwd
/home/usuario/projetos/linux

# pwd lógico (respeita links simbólicos)
$ pwd -L
/home/usuario/link_para_projetos

# pwd físico (resolve links simbólicos)
$ pwd -P
/home/usuario/projetos/linux
```

Dica

Seu prompt (PS1) geralmente já mostra o diretório atual, mas `pwd` é útil em scripts onde você precisa capturar o caminho em uma variável: `DIR_ATUAL=$(pwd)`

2.2 ls — List Directory Contents

O comando `ls` é um dos mais usados no terminal. Lista arquivos e diretórios com diversas opções de formatação.

2.2.1 Opções Essenciais

```
# Listagem básica
$ ls
arquivo1.txt  diretorio/  script.sh

# Listagem detalhada (long format)
$ ls -l
drwxr-xr-x 2 usuario grupo 4096 Jun 14 10:00 diretorio
-rw-r--r-- 1 usuario grupo 1234 Jun 14 09:30 arquivo.txt
-rwxr-xr-x 1 usuario grupo  512 Jun 14 08:00 script.sh

# Listagem com tamanhos human-readable
$ ls -lh
-rw-r--r-- 1 usuario grupo 1.2K Jun 14 09:30 arquivo.txt
-rw-r--r-- 1 usuario grupo 2.4M Jun 14 09:45 video.mp4
```



```
# Mostrar arquivos ocultos (começam com .)
$ ls -la
.  .. .bashrc .profile arquivo.txt diretorio/

# Ordenar por data de modificação (mais recente primeiro)
$ ls -lt

# Ordenar por data de acesso
$ ls -ltu

# Ordenar por tamanho (maior primeiro)
$ ls -lhS

# Ordenar reverso
$ ls -ltr # mais antigo primeiro

# Mostrar tipo de arquivo (indicador no final)
$ ls -F
arquivo.txt diretorio/ link_simbolico@ executavel*

# Listar recursivamente
$ ls -R

# Listar com inode number
$ ls -li
123456 arquivo.txt

# Listar apenas diretórios
$ ls -d */

# Listar com cores (auto-detecta terminal)
$ ls --color=auto

# Mostrar UID/GID numéricos
$ ls -ln

# Mostrar tempo completo
$ ls -l --time-style=full-iso
```

2.2.2 Combinações Poderosas

```
# Listar todos os arquivos, ordenados por tamanho, com tamanhos legíveis
$ ls -lahS

# Listar arquivos modificados nas últimas 24 horas
$ ls -lt --time-style=full-iso | head -10

# Listar apenas arquivos executáveis
$ ls -l | grep '^-..x'

# Contar número de arquivos em um diretório
$ ls -l | wc -l

# Listar arquivos com extensão .txt
$ ls -l *.txt

# Listar arquivos que NÃO são .txt
$ ls -l | grep -v '\.txt$'
```

⚠ Cuidado com ls em scripts

Nunca use `ls` para processar arquivos em scripts (especialmente nomes com espaços). Use `find` com `-print0` ou globbing seguro. `ls` é para visualização humana, não para processamento automatizado.

2.3 cd — Change Directory

O comando `cd` muda o diretório atual. É um comando interno do shell (built-in), não um programa externo.

```
# Ir para um diretório específico
$ cd /home/usuario/projetos

# Ir para o diretório home
$ cd
$ cd ~
$ cd ~usuario

# Ir para o diretório anterior
$ cd -

# Subir um nível
$ cd ..

# Subir dois níveis
$ cd ../../..

# Ir para um diretório relativo
$ cd projetos/linux

# Ir para diretório absoluto
$ cd /var/log

# Usar tab completion
$ cd /u/l/b<TAB> → /usr/local/bin

# Usar variável de ambiente
$ cd $HOME
$ cd $PWD

# CDPATH – caminhos de busca para cd
$ export CDPATH=.:~/projetos:/var/www
$ cd site # procura em ., ~/projetos, /var/www
```

2.3.1 Atalhos Úteis

Atalho	Significado
<code>cd</code> ou <code>cd ~</code>	Diretório home do usuário atual
<code>cd -</code>	Diretório anterior (último <code>cd</code>)
<code>cd ..</code>	Diretório pai (um nível acima)
<code>cd ../../..</code>	Dois níveis acima

```
cd /
```

Raiz do sistema

```
cd ~usuario
```

Home de outro usuário

2.4 mkdir — Make Directory

Cria diretórios. Parece simples, mas tem opções poderosas.

```
# Criar um diretório
$ mkdir novo_diretorio

# Criar múltiplos diretórios
$ mkdir dir1 dir2 dir3

# Criar diretórios aninhados (parents)
$ mkdir -p projetos/2024/junho/relatorios
# Cria toda a hierarquia se não existir

# Criar com permissões específicas
$ mkdir -m 755 diretorio_publico

# Criar diretório e verificar
$ mkdir -v novo_dir
mkdir: created directory 'novo_dir'

# Criar estrutura de projeto completa
$ mkdir -p meu_projeto/{src,include,lib,tests,docs,bin}
# Cria: meu_projeto/src, meu_projeto/include, etc.

# Criar diretório com data atual no nome
$ mkdir backup_$(date +%Y-%m-%d)
# Resultado: backup_2024-06-14
```

Brace Expansion

O bash suporta **brace expansion** para criar múltiplos itens de forma concisa:

```
$ mkdir -p projeto/{src,tests,docs}/{2023,2024}
# Cria: projeto/src/2023, projeto/src/2024, projeto/tests/2023, ...
```

2.5 rmdir — Remove Directory

Remove diretórios **vazios**. Para diretórios com conteúdo, use `rm -r` (com cuidado!).

```
# Remover diretório vazio
$ rmdir diretorio_vazio

# Remover múltiplos diretórios vazios
$ rmdir dir1 dir2 dir3

# Remover diretório e seus pais se ficarem vazios
$ rmdir -p projeto/2024/junho
```

```
# Remove junho/, se 2024/ ficar vazio remove também, etc.

# Remover diretórios vazios recursivamente
$ find . -type d -empty -delete

# Verificar antes de remover
$ rmdir -v diretorio
rmdir: removing directory, 'diretorio'
```

rmdir vs. rm -r

rmdir só remove diretórios vazios — é seguro. **rm -r** remove diretórios e TODO o conteúdo. Use **rm -ri** (interativo) ou **rm -rI** (pergunta uma vez para muitos arquivos) para evitar acidentes.

2.6 touch — Change File Timestamps

O **touch** é usado para criar arquivos vazios ou atualizar timestamps. É mais poderoso do que parece.

```
# Criar arquivo vazio
$ touch novo_arquivo.txt

# Atualizar timestamp de acesso e modificação para agora
$ touch arquivo_existente.txt

# Atualizar apenas timestamp de acesso
$ touch -a arquivo.txt

# Atualizar apenas timestamp de modificação
$ touch -m arquivo.txt

# Não criar arquivo se não existir
$ touch -c arquivo_inexistente.txt

# Usar timestamp de outro arquivo como referência
$ touch -r arquivo_referencia.txt arquivo_alvo.txt

# Definir timestamp específico
$ touch -t 202406141200.00 arquivo.txt
# Formato: [[CC]YY]MMDDhhmm[.ss]
# 2024-06-14 12:00:00

# Definir data com string legível
$ touch -d "2024-06-14 12:00:00" arquivo.txt
$ touch -d "yesterday" arquivo.txt
$ touch -d "2 days ago" arquivo.txt
$ touch -d "next Monday" arquivo.txt

# Criar múltiplos arquivos
$ touch arquivo{1..10}.txt
# Cria: arquivo1.txt, arquivo2.txt, ..., arquivo10.txt

# Criar arquivo com espaço no nome (use aspas)
$ touch "meu arquivo.txt"

# Criar arquivo oculto
$ touch .config
```

2.6.1 Timestamps no Linux

Timestamp	Significado	Comando
atime	Último acesso (leitura)	<code>touch -a</code> , <code>ls -lu</code>
mtime	Última modificação do conteúdo	<code>touch -m</code> , <code>ls -l</code>
ctime	Última mudança nos metadados (permissões, dono)	<code>ls -lc</code>

2.7 cp — Copy Files and Directories

Copia arquivos e diretórios. Um dos comandos mais perigosos se usado sem atenção.

```
# Copiar arquivo
$ cp origem.txt destino.txt

# Copiar para diretório
$ cp arquivo.txt /backup/

# Copiar múltiplos arquivos
$ cp arquivo1.txt arquivo2.txt /destino/

# Copiar com preservação de atributos
$ cp -p arquivo.txt backup/
# Preserva: modo, ownership, timestamps

# Copiar recursivamente (diretórios)
$ cp -r diretorio/ /backup/

# Copiar recursivamente com preservação completa
$ cp -a diretorio/ /backup/
# Equivalente a -dR --preserve=all

# Copiar com verificação de sobrescrita (interativo)
$ cp -i arquivo.txt /destino/

# Forçar sobrescrita sem perguntar
$ cp -f arquivo.txt /destino/

# Copiar apenas se o arquivo for mais novo
$ cp -u arquivo.txt /destino/

# Copiar com barra de progresso (pv)
$ cp arquivo_grande.iso /destino/ & pv -pteba arquivo_grande.iso > /destino/arquivo_grande.iso

# Copiar com backup automático
$ cp --backup=numbered arquivo.txt /destino/
# Cria: arquivo.txt.~1~, arquivo.txt.~2~, etc.

# Copiar links simbólicos como links (não seguir)
$ cp -d link_simbolico /destino/

# Criar link hard em vez de copiar
$ cp -l arquivo.txt /destino/
```

```
# Copiar com sparse (arquivos esparsos)
$ cp --sparse=always arquivo.img /destino/
```

Cuidado com cp e diretórios

`cp -r` copia recursivamente. Se o destino já existe e é um diretório, o conteúdo é copiado *dentro* dele. Se não existe, cria o diretório. Verifique sempre com `ls` antes.

2.8 mv — Move/Rename Files

Move ou renomeia arquivos e diretórios. No Linux, renomear e mover são a mesma operação (ambos alteram o path).

```
# Renomear arquivo
$ mv antigo.txt novo.txt

# Mover arquivo para diretório
$ mv arquivo.txt /destino/

# Mover múltiplos arquivos
$ mv *.txt /backup/

# Mover e sobrescrever sem perguntar
$ mv -f arquivo.txt /destino/

# Mover com confirmação (interativo)
$ mv -i arquivo.txt /destino/

# Mover apenas se o destino for mais antigo
$ mv -u arquivo.txt /destino/

# Mover com backup
$ mv --backup=numbered arquivo.txt /destino/

# Mover diretório inteiro
$ mv diretorio/ /novo/local/

# Renomear com substituição de padrão (bash)
$ for f in *.txt; do mv "$f" "${f%.txt}.bak"; done

# Renomear múltiplos arquivos com rename (perl)
$ rename 's/\.txt$/bak/' *.txt

# Mover arquivos modificados hoje
$ find . -mtime -1 -type f -exec mv {} /backup/hoje/ \;
```

mv é atômico para arquivos no mesmo filesystem

Quando `mv` move um arquivo dentro do mesmo filesystem, ele apenas atualiza os metadados no inode — os dados do arquivo não são copiados. Isso é instantâneo e atômico, mesmo para arquivos gigantes.

2.9 rm — Remove Files

Remove arquivos. **PERMANENTEMENTE**. Não há lixeira no terminal. Use com extrema cautela.

```
# Remover arquivo
$ rm arquivo.txt

# Remover múltiplos arquivos
$ rm arquivo1.txt arquivo2.txt

# Remover com confirmação
$ rm -i arquivo.txt

# Remover sem confirmação (padrão para root!)
$ rm -f arquivo.txt

# Remover diretório e conteúdo recursivamente
$ rm -r diretorio/

# Remover diretório vazio
$ rm -d diretorio_vazio/
# Equivalente a rmdir

# Remover recursivamente e forçado (MUITO CUIDADO!)
$ rm -rf diretorio/

# Remover arquivos com nomes estranhos
$ rm -- -arquivo_com_hifen_no_inicio
$ rm ./-arquivo_com_hifen

# Remover exceto um arquivo
$ rm !(arquivo_importante.txt)
# Requer: shopt -s extglob

# Remover arquivos antigos
$ find /tmp -type f -mtime +7 -delete

# Remover com log
$ rm -v arquivo.txt
removed 'arquivo.txt'

# Simular remoção (não remove, apenas mostra o que faria)
# Use echo ou rm -i combinado com find
$ find . -name "*.tmp" -exec echo rm {} \;
```

O Comando Mais Perigoso do Linux

`rm -rf /` apaga TODO o sistema. Algumas distribuições modernas têm proteção (`rm --preserve-root` é padrão), mas `rm -rf /*` ou `rm -rf / --no-preserve-root` ainda funcionam. Sempre verifique o caminho antes de executar `rm -rf`.

Regra: Nunca use `rm -rf` com variáveis não verificadas ou com wildcards sem testar primeiro.

2.10 cat — Concatenate and Print

O `cat` concatena e exibe arquivos. É frequentemente usado para visualizar arquivos pequenos, mas tem usos mais avançados.

```
# Exibir conteúdo de arquivo
$ cat arquivo.txt

# Concatenar múltiplos arquivos
$ cat arquivo1.txt arquivo2.txt > combinado.txt

# Concatenar com numeração de linhas
$ cat -n arquivo.txt

# Numeração apenas para linhas não-vazias
$ cat -b arquivo.txt

# Mostrar caracteres especiais (tabs, fim de linha)
$ cat -A arquivo.txt
# ^I = tab, $ = fim de linha

# Suprimir linhas repetidas em branco
$ cat -s arquivo.txt

# Criar arquivo com heredoc
$ cat > arquivo.txt << 'EOF'
Linha 1
Linha 2
Linha 3
EOF

# Criar arquivo com conteúdo inline
$ cat > config.ini < arquivo.txt
# Equivalente a: > arquivo.txt
```

⚠ Useless Use of Cat (UUOC)

Usar `cat arquivo | comando` quando `comando arquivo` funciona é considerado um anti-pattern. O `cat` cria um processo desnecessário. Prefira `grep "padrao" arquivo.txt` em vez de `cat arquivo.txt | grep "padrao"`.

2.11 tac — Concatenate and Print in Reverse

O `tac` é o inverso do `cat`: exibe o arquivo de trás para frente (por linhas).

```
# Exibir arquivo invertido (linhas)
$ tac arquivo.txt

# Usar separador customizado
$ tac -s "---" arquivo.txt

# Processar logs em ordem inversa (mais recente primeiro)
$ tac /var/log/syslog | head -20

# Combinar com grep para buscar do final
$ tac arquivo.log | grep "ERRO" | head -5
```


2.12 nl — Number Lines

O **nl** numera linhas de arquivo com mais controle que **cat -n**.

```
# Numerar todas as linhas
$ nl arquivo.txt

# Numerar apenas linhas não-vazias (padrão)
$ nl -b t arquivo.txt

# Numerar todas as linhas
$ nl -b a arquivo.txt

# Numerar apenas linhas com texto
$ nl -b n arquivo.txt

# Definir formato de numeração
$ nl -n rz arquivo.txt    # zeros à esquerda
$ nl -n ln arquivo.txt    # alinhado à esquerda
$ nl -n rn arquivo.txt    # alinhado à direita (padrão)

# Definir largura do número
$ nl -w 5 arquivo.txt

# Definir string de separação
$ nl -s ". " arquivo.txt
# 1. Linha um
# 2. Linha dois

# Usar delimitador de seção lógica
$ nl -d "@@" arquivo.txt
```

2.13 less — Paginator Avançado

O **less** é um paginador que permite navegar por arquivos grandes de forma interativa. É o sucessor do **more** e muito mais poderoso.

```
# Visualizar arquivo
$ less arquivo.txt

# Visualizar com numeração de linhas
$ less -N arquivo.txt

# Visualizar com highlight de busca
$ less -g arquivo.txt

# Ignorar case na busca
$ less -I arquivo.txt

# Visualizar múltiplos arquivos
$ less arquivo1.txt arquivo2.txt
# :n → próximo arquivo
# :p → arquivo anterior

# Visualizar saída de comando
$ dmesg | less
$ ps aux | less
```

```
# Buscar dentro do less
# /padrao    → buscar para frente
# ?padrao    → buscar para trás
# n          → próxima ocorrência
# N          → ocorrência anterior

# Navegação no less
# Space/PageDown → página abaixo
# b/PageUp      → página acima
# j/↓           → uma linha abaixo
# k/↑           → uma linha acima
# g             → início do arquivo
# G             → fim do arquivo
# 50g           → linha 50
# %50           → 50% do arquivo
# q            → sair

# Seguir arquivo em tempo real (como tail -f)
$ less +F arquivo.log
# Ctrl+C para pausar, F para continuar

# Editar arquivo diretamente do less
# v → abre no editor padrão ($VISUAL ou $EDITOR)
```

less é mais que more

less permite navegar para trás (ao contrário de **more**), buscar, marcar posições, e visualizar arquivos sem carregá-los completamente na memória. Para arquivos grandes, **less** é essencial.

2.14 more — Paginador Simples

O **more** é um paginador básico. Na maioria dos casos, prefira **less**.

```
# Visualizar arquivo
$ more arquivo.txt

# Definir número de linhas por tela
$ more -10 arquivo.txt

# Desabilitar scroll
$ more -d arquivo.txt

# Navegação no more
# Space → próxima página
# Enter → próxima linha
# b      → página anterior (nem sempre funciona)
# /padrao → buscar
# q      → sair
```

2.15 head — Output First Part

Exibe as primeiras linhas de um arquivo. Útil para pré-visualizar arquivos grandes.

```
# Primeiras 10 linhas (padrão)
$ head arquivo.txt

# Primeiras 20 linhas
$ head -n 20 arquivo.txt
$ head -20 arquivo.txt

# Primeiros 100 bytes
$ head -c 100 arquivo.txt

# Primeiras linhas de múltiplos arquivos
$ head arquivo1.txt arquivo2.txt

# Combinar com pipe
$ cat /var/log/syslog | head -50
# Melhor: head -50 /var/log/syslog

# Ver cabeçalho de arquivo CSV
$ head -1 dados.csv

# Ver primeiras linhas de todos os arquivos .log
$ head -n 5 *.log
```

2.16 tail — Output Last Part

Exibe as últimas linhas de um arquivo. Essencial para monitorar logs em tempo real.

```
# Últimas 10 linhas
$ tail arquivo.txt

# Últimas 20 linhas
$ tail -n 20 arquivo.txt

# Últimos 100 bytes
$ tail -c 100 arquivo.txt

# Seguir arquivo em tempo real (follow)
$ tail -f /var/log/syslog

# Seguir e reabrir se o arquivo for rotacionado
$ tail -F /var/log/syslog
# Útil para logs que são renomeados/compactados

# Combinar head e tail para extrair linhas do meio
$ head -n 100 arquivo.txt | tail -n 20
# Linhas 81-100

# Monitorar múltiplos arquivos
$ tail -f /var/log/syslog /var/log/auth.log

# Ver logs desde uma linha específica
$ tail -n +50 arquivo.txt
# Mostra a partir da linha 50

# Usar com grep para filtrar logs em tempo real
$ tail -f /var/log/syslog | grep "ERRO"
```

✓ **tail -f** é indispensável para sysadmins

Quando um serviço está com problemas, `tail -f /var/log/servico.log` permite ver os erros em tempo real. Combine com `grep` para filtrar apenas o que interessa.

2.17 file — Determine File Type

O `file` identifica o tipo de arquivo analisando seu conteúdo, não apenas a extensão.

```
# Identificar tipo de arquivo
$ file arquivo.txt
arquivo.txt: ASCII text

$ file programa
programa: ELF 64-bit LSB executable, x86-64, version 1 (SYSV)

$ file imagem.png
imagem.png: PNG image data, 800 x 600, 8-bit/color RGBA

# Identificar múltiplos arquivos
$ file *

# Identificar com mais detalhes
$ file -i arquivo.txt
arquivo.txt: text/plain; charset=us-ascii

# Ver magic number em hexadecimal
$ file -k arquivo_desconhecido

# Listar todos os tipos de arquivo conhecidos
$ file -l

# Identificar arquivo dentro de arquivo compactado
$ file -z arquivo.tar.gz

# Verificar arquivo de core dump
$ file core
```

Como file funciona?

O `file` usa um banco de dados de "magic numbers" (bytes iniciais característicos) localizado em `/usr/share/file/magic`. Por exemplo, arquivos ELF começam com `0x7F 'E' 'L' 'F'`, arquivos PNG com `0x89 'P' 'N' 'G'`.

2.18 stat — Display File Status

O `stat` exibe informações detalhadas sobre arquivos — muito mais que `ls -l`.

```
# Informações completas do arquivo
$ stat arquivo.txt
File: arquivo.txt
```

```
Size: 1234      Blocks: 8      IO Block: 4096   regular file
Device: 801h/2049d Inode: 131073    Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/ usuario)   Gid: ( 1000/ grupo)
Access: 2024-06-14 10:00:00.000000000 -0300
Modify: 2024-06-14 09:30:00.000000000 -0300
Change: 2024-06-14 09:30:00.000000000 -0300
Birth: 2024-06-14 08:00:00.000000000 -0300
```

```
# Formato customizado
```

```
$ stat -c "%n: %s bytes, permissões %a" arquivo.txt
arquivo.txt: 1234 bytes, permissões 644
```

```
# Formatos úteis
```

```
$ stat -c "%y %n" *.txt      # data de modificação + nome
$ stat -c "%s %n" *         # tamanho + nome
$ stat -c "%U %G %a %n" *   # dono, grupo, permissões, nome
```

```
# Ver sistema de arquivos do arquivo
```

```
$ stat -f .
File: "."
ID: 1234567890   Namelen: 255   Type: ext2/ext3
Block size: 4096   Fundamental block size: 4096
Blocks: Total: 1000000 Free: 500000 Available: 450000
Inodes: Total: 256000 Free: 200000
```

```
# Ver informações de link simbólico (não seguir)
```

```
$ stat -c "%N" link_simbolico
'link_simbolico' -> 'arquivo_original.txt'
```

2.18.1 Formatos de saída do stat

Formato	Descrição
%n	Nome do arquivo
%s	Tamanho em bytes
%b	Número de blocos alocados
%o	Tamanho do bloco de I/O
%i	Número do inode
%a	Permissões em octal
%A	Permissões em formato humano
%U	Nome do dono
%u	UID do dono
%G	Nome do grupo
%g	GID do grupo
%x	Data de acesso

%y	Data de modificação
%z	Data de mudança (ctime)
%w	Data de criação (birth)
%N	Nome com link simbólico resolvido

2.19 tree — Display Directory Tree

O **tree** exibe a estrutura de diretórios em formato de árvore. Não vem instalado por padrão em todas as distribuições.

```
# Instalar tree
$ sudo apt install tree      # Debian/Ubuntu
$ sudo dnf install tree     # Fedora
$ sudo pacman -S tree       # Arch

# Exibir árvore do diretório atual
$ tree

.
├── diretorio1
│   ├── arquivo1.txt
│   └── arquivo2.txt
├── diretorio2
│   └── subdir
│       └── arquivo3.txt
└── arquivo_raiz.txt

# Limitar profundidade
$ tree -L 2

# Mostrar apenas diretórios
$ tree -d

# Mostrar arquivos com padrão específico
$ tree -P "*.c"

# Ignorar padrão
$ tree -I "node_modules|__pycache__"

# Mostrar tamanhos
$ tree -h

# Mostrar permissões
$ tree -p

# Mostrar inode numbers
$ tree -i

# Salvar árvore em arquivo
$ tree > estrutura.txt

# Colorir saída
$ tree -C

# Mostrar arquivos ocultos
$ tree -a
```

```
# Ordenar por versão (natural)
$ tree -v

# Usar caracteres ASCII (não Unicode)
$ tree -A
```

tree como documentação

Use `tree -L 2 -I 'node_modules|*.pyc' > ESTRUTURA.md` para documentar a estrutura de um projeto em READMEs.

2.20 echo — Display a Line of Text

O `echo` exibe texto. Simples, mas essencial para scripts e testes.

```
# Exibir texto simples
$ echo "Hello, World!"

# Exibir sem nova linha no final
$ echo -n "Sem nova linha"

# Exibir com interpretação de escapes
$ echo -e "Linha 1
Linha 2 Tabulado"

# Escapes comuns
#
#   → nova linha
#       → tabulação
# \ → barra invertida
#   → beep (alert)
#
#   → carriage return
#   → backspace

# Exibir valor de variável
$ echo $HOME
$ echo $PATH
$ echo $?

# Exibir resultado de comando
$ echo "Hoje é $(date +%Y-%m-%d)"

# Criar arquivo com echo
$ echo "conteúdo" > arquivo.txt

# Anexar a arquivo
$ echo "nova linha" >> arquivo.txt

# Exibir com aspas simples (não expande variáveis)
$ echo '$HOME'
$HOME

# Exibir com aspas duplas (expande variáveis)
$ echo "$HOME"
```

```
/home/usuario

# Exibir asterisco literal
$ echo "*"
*
$ echo *
# (lista todos os arquivos do diretório!)
```

2.21 printf — Format and Print Data

O `printf` é mais poderoso que `echo`, com formatação tipo C. É um built-in do bash e também um comando externo.

```
# Formatação básica
$ printf "Hello, %s!"
"World"

# Múltiplos argumentos
$ printf "Nome: %s, Idade: %d"
"João" 25

# Formatação de números
$ printf "Decimal: %d"
"42"
$ printf "Hex: %x"
"255"
$ printf "Octal: %o"
"8"
$ printf "Float: %.2f"
"3.14159"

# Alinhamento
$ printf "|%-10s|%10s|"
"Esquerda" "Direita"
# |Esquerda |      Direita|

# Preencher com zeros
$ printf "%010d"
"42"
# 0000000042

# Formatação de data
$ printf "%(%Y-%m-%d %H:%M:%S)T"
"-1"
# 2024-06-14 12:00:00

# Criar tabela formatada
$ printf "%-15s %10s %10s"
"ARQUIVO" "TAMANHO" "PERMISSÕES"
$ printf "%-15s %10d %10s"
"arquivo.txt" 1234 "-rw-r--r--"

# Usar em loop
$ for f in *.txt; do printf "Processando: %s"
" $f"; done

# Imprimir sem nova linha
$ printf "Carregando"
$ printf "."
```



```
$ printf ".  
$ printf ".  
"
```

2.21.1 Especificadores de Formato

Especificador	Descrição
<code>%s</code>	String
<code>%d</code> ou <code>%i</code>	Inteiro decimal
<code>%u</code>	Inteiro unsigned
<code>%x</code>	Hexadecimal (minúsculo)
<code>%X</code>	Hexadecimal (maiúsculo)
<code>%o</code>	Octal
<code>%f</code>	Ponto flutuante
<code>%e</code>	Notação científica
<code>%c</code>	Caractere único
<code>%%</code>	Literal %

🔧 Exercícios — Volume 2

1. Use `ls -la` no seu home e identifique: quantos arquivos ocultos existem? Qual o maior arquivo? Qual o mais recente?
2. Crie a estrutura `projeto/{src,include,lib,tests,docs}` com um único comando. Use `tree` para verificar.
3. Crie 20 arquivos vazios com nomes `teste_01.txt` a `teste_20.txt` usando brace expansion.
4. Use `cat` com heredoc para criar um arquivo de configuração com 10 linhas.
5. Use `head` e `tail` para extrair as linhas 50-60 de `/etc/passwd`.
6. Execute `file /bin/ls` e interprete TODOS os campos da saída. O que é "ELF"? "LSB"? "dynamically linked"?
7. Use `stat` para mostrar: nome, tamanho, inode, dono e permissões de todos os arquivos de `/bin`.
8. Monitore `/var/log/syslog` com `tail -f` em uma janela e gere eventos em outra. Use `grep` para filtrar.
9. Crie um script que use `printf` para formatar uma tabela com 5 arquivos, mostrando nome, tamanho e permissões alinhados.
10. Use `less` para abrir `/var/log/syslog`, busque por "error", navegue entre ocorrências e saia.



Volume 3 — Busca e Manipulação de Texto

grep, find, sed, awk e as ferramentas que transformam dados em informação.

3.1 grep — Global Regular Expression Print

O **grep** busca padrões em texto usando expressões regulares. É uma das ferramentas mais poderosas do terminal.

3.1.1 Uso Básico

```
# Buscar palavra em arquivo
$ grep "erro" log.txt

# Buscar em múltiplos arquivos
$ grep "erro" *.log

# Buscar recursivamente em diretórios
$ grep -r "erro" /var/log/

# Buscar ignorando case
$ grep -i "erro" log.txt
# Encontra: erro, ERRO, Erro, eRrO

# Inverter match (linhas que NÃO contêm o padrão)
$ grep -v "sucesso" log.txt

# Mostrar número da linha
$ grep -n "erro" log.txt

# Mostrar apenas contagem
$ grep -c "erro" log.txt

# Mostrar nome do arquivo (com múltiplos arquivos)
$ grep -l "erro" *.log

# Mostrar linhas que NÃO contêm o padrão
$ grep -L "erro" *.log

# Mostrar contexto (linhas ao redor)
$ grep -C 2 "erro" log.txt # 2 linhas antes e depois
$ grep -B 2 "erro" log.txt # 2 linhas antes
$ grep -A 2 "erro" log.txt # 2 linhas depois
```

3.1.2 Expressões Regulares com grep

```
# Buscar no início da linha (^)
$ grep "^erro" log.txt

# Buscar no final da linha ($)
$ grep "erro$" log.txt
```

```

# Qualquer caractere (.)
$ grep "e.r.o" log.txt
# Encontra: erro, e1r2o, e r o

# Conjunto de caracteres ([])
$ grep "[Ee]rro" log.txt
# Encontra: erro, Erro

# Range de caracteres
$ grep "[a-z]erro" log.txt
# Encontra: aerro, berro, ..., zerro

# Negar conjunto
$ grep "[^0-9]erro" log.txt
# Encontra erro precedido por algo que nao eh numero

# Quantificadores
$ grep "erro*" log.txt      # 0 ou mais 'r'
$ grep "erro+" log.txt     # 1 ou mais 'r' (grep -E)
$ grep "erro?" log.txt     # 0 ou 1 'r' (grep -E)
$ grep "erro{2,3}" log.txt # 2 a 3 'r' (grep -E)

# Alternativa (OR)
$ grep -E "erro|falha" log.txt

# Grupos
$ grep -E "(erro|falha) critica" log.txt

# Palavra completa (\b)
$ grep -w "erro" log.txt
# Nao encontra "erroses", apenas "erro" isolado

# grep estendido (ERE)
$ grep -E "padrao+" arquivo.txt
# ou
$ egrep "padrao+" arquivo.txt

```

3.1.3 grep Avancado

```

# Buscar apenas palavras completas
$ grep -w "root" /etc/passwd

# Limitar numero de matches
$ grep -m 5 "erro" log.txt

# Buscar em arquivo binario (tratar como texto)
$ grep -a "texto" arquivo.bin

# Mostrar posicao do match
$ grep -o "erro" log.txt
# Apenas o texto matchado, nao a linha inteira

# Buscar com arquivo de padroes
$ grep -f padroes.txt log.txt
# padroes.txt contem um padrao por linha

# Excluir arquivos/diretorios na busca recursiva
$ grep -r "erro" /var/log/ --exclude="*.gz"
$ grep -r "erro" /var/log/ --exclude-dir="archive"

```

```
# Colorir output
$ grep --color=auto "erro" log.txt

# Contar ocorrencias por arquivo
$ grep -c "erro" *.log

# Mostrar apenas arquivos SEM match
$ grep -L "erro" *.log

# Usar com process substitution
$ grep "erro" <(dmesg)

# grep com Perl regex (PCRE)
$ grep -P "\d{3}-\d{2}-\d{4}" arquivo.txt
# Busca SSN: 123-45-6789
```

grep vs. egrep vs. fgrep

- **grep** : Basico (BRE - Basic Regular Expressions)
- **egrep** ou **grep -E** : Estendido (ERE - Extended Regular Expressions)
- **fgrep** ou **grep -F** : Fixed strings (sem regex, busca literal)
- **grep -P** : Perl Compatible Regular Expressions (PCRE)

3.2 find — Search for Files

O **find** busca arquivos e diretorios no sistema de arquivos. E extremamente poderoso e flexivel.

3.2.1 Busca Basica

```
# Buscar por nome
$ find /home -name "*.txt"

# Buscar ignorando case
$ find /home -iname "*.TXT"

# Buscar por tipo
$ find /home -type f      # arquivos regulares
$ find /home -type d      # diretorios
$ find /home -type l      # links simbolicos
$ find /home -type b      # dispositivos de bloco
$ find /home -type c      # dispositivos de caractere
$ find /home -type p      # named pipes
$ find /home -type s      # sockets

# Buscar por tamanho
$ find /home -size +100M   # maior que 100MB
$ find /home -size -1k     # menor que 1KB
$ find /home -size 10M     # exatamente 10MB
$ find /home -size +1G -size -5G # entre 1GB e 5GB

# Buscar por tempo de modificacao
$ find /home -mtime -1     # modificado nas ultimas 24h
$ find /home -mtime +7     # modificado ha mais de 7 dias
$ find /home -mtime 1      # modificado entre 24h e 48h atras
```

```
# Buscar por tempo de acesso
$ find /home -atime +30      # nao acessado ha 30+ dias

# Buscar por tempo de mudanca (ctime)
$ find /home -ctime -1

# Buscar por minutos (mais preciso)
$ find /home -mmin -60      # modificado na ultima hora
```

3.2.2 Operadores Logicos

```
# AND (padrao)
$ find /home -name "*.txt" -size +1M

# OR
$ find /home \( -name "*.txt" -o -name "*.log" \)

# NOT
$ find /home ! -name "*.txt"

# Combinados
$ find /home -type f \( -name "*.tmp" -o -name "*.bak" \) -size +10M

# Precedencia (use parenteses)
$ find /home \( -type f -a -name "*.log" \) -o -type d
```

3.2.3 Acoes com find

```
# Executar comando para cada arquivo encontrado
$ find /home -name "*.log" -exec ls -lh {} \;

# Executar com multiplos arquivos de uma vez (mais eficiente)
$ find /home -name "*.log" -exec ls -lh {} +

# Deletar arquivos encontrados
$ find /tmp -name "*.tmp" -type f -delete

# Ou com exec
$ find /tmp -name "*.tmp" -type f -exec rm {} \;

# Copiar arquivos encontrados
$ find /home -name "*.conf" -exec cp {} /backup/ \;

# Mover arquivos
$ find /home -name "*.old" -exec mv {} /arquivo/ \;

# Executar script
$ find /home -name "*.sh" -exec bash {} \;

# Chmod em arquivos encontrados
$ find /home -type f -exec chmod 644 {} \;
$ find /home -type d -exec chmod 755 {} \;

# Usar -ok para confirmacao interativa
$ find /home -name "*.txt" -ok rm {} \;
```

3.2.4 find Avancado

```
# Buscar por permissões
$ find /home -perm 644
$ find /home -perm /u=s      # SUID
$ find /home -perm -g=w     # grupo tem escrita

# Buscar por dono
$ find /home -user root
$ find /home -group sudo

# Buscar por inode
$ find / -inum 123456

# Buscar links para um arquivo específico
$ find / -samefile /path/to/file

# Buscar arquivos vazios
$ find /home -type f -empty

# Buscar diretórios vazios
$ find /home -type d -empty

# Limitar profundidade
$ find /home -maxdepth 2 -name "*.txt"
$ find /home -mindepth 2 -name "*.txt"

# Não seguir links simbólicos (padrão)
$ find /home -name "*.txt"

# Seguir links simbólicos
$ find /home -follow -name "*.txt"

# Buscar em múltiplos diretórios
$ find /home /var /tmp -name "*.log"

# Excluir diretórios da busca
$ find /home -path "*/node_modules" -prune -o -name "*.js" -print

# Usar -printf para formatar saída
$ find /home -type f -printf "%p %s %TY-%Tm-%Td\n"
# path, size, data

# Buscar com regex (caminho completo)
$ find /home -regex ".*\.txt$"

# Buscar com regex ignorando case
$ find /home -iregex ".*\.TXT$"

```

✓ find + xargs é mais eficiente que find -exec

Para processar muitos arquivos, `find ... | xargs comando` é mais eficiente que `find ... -exec comando {} \;` porque `xargs` passa múltiplos arquivos de uma vez. Use `find ... -print0 | xargs -0 comando` para nomes com espaços.

3.3 locate — Find Files by Name

O `locate` busca em um banco de dados pre-construido, sendo muito mais rapido que `find` para buscas por nome. O banco de dados eh atualizado pelo `updatedb`.

```
# Instalar mlocate
$ sudo apt install mlocate

# Buscar arquivo por nome
$ locate arquivo.txt

# Buscar ignorando case
$ locate -i arquivo.txt

# Mostrar apenas N resultados
$ locate -n 10 arquivo.txt

# Mostrar arquivos existentes (verifica se existem)
$ locate -e arquivo.txt

# Atualizar banco de dados manualmente
$ sudo updatedb

# Buscar com regex
$ locate -r "\.conf$"

# Buscar arquivos de um usuario especifico
$ locate -u usuario arquivo.txt

# Estatisticas do banco de dados
$ locate -S
# Mostra: arquivos, diretorios, tamanho do DB, etc.

# Buscar apenas arquivos (nao diretorios)
$ locate -r "\.txt$"

# Excluir padroes do updatedb
# Edite /etc/updatedb.conf para excluir /tmp, /proc, etc.
```

⚠ `locate` nao encontra arquivos recém-criados

O banco de dados do `locate` eh atualizado periodicamente (geralmente uma vez por dia via cron). Arquivos criados apos a ultima atualizacao nao aparecerao. Use `find` para buscas em tempo real.

3.4 which — Locate a Command

O `which` mostra o caminho completo do executavel que seria executado se voce digitasse o comando.

```
# Localizar comando
$ which ls
/bin/ls

$ which python3
/usr/bin/python3

# Mostrar todos os caminhos (se houver aliases)
```

```
$ which -a python
/usr/bin/python
/usr/local/bin/python

# Verificar se comando existe
$ which nginx
# (sem saida = nao encontrado)

# Usar em scripts
$ if which nginx >/dev/null; then echo "nginx instalado"; fi
```

3.5 whereis — Locate Binary, Source, and Manual Page

O **whereis** busca binarios, codigo-fonte e man pages de um comando.

```
# Localizar tudo sobre um comando
$ whereis ls
ls: /bin/ls /usr/share/man/man1/ls.1.gz

# Buscar apenas binarios
$ whereis -b ls

# Buscar apenas man pages
$ whereis -m ls

# Buscar apenas codigo-fonte
$ whereis -s ls

# Buscar em caminhos nao padrao
$ whereis -B /usr/local/bin -f ls

# Limitar resultados
$ whereis -l ls

# Mostra os caminhos de busca efetivos
```

3.6 sort — Sort Lines of Text

O **sort** ordena linhas de texto. Suporta ordenacao numerica, por data, por campo, e mais.

```
# Ordenar alfabeticamente
$ sort arquivo.txt

# Ordenar inversamente
$ sort -r arquivo.txt

# Ordenar numericamente
$ sort -n numeros.txt

# Ordenar por tamanho (human-readable)
$ sort -h tamanhos.txt

# Ordenar ignorando case
$ sort -f arquivo.txt

# Ordenar removendo duplicatas
```



```
$ sort -u arquivo.txt

# Ordenar por campo especifico
$ sort -k 3 arquivo.txt      # 3o campo (padrao: tab/space)
$ sort -k 2,2n arquivo.txt   # 2o campo, numerico
$ sort -t: -k 3n /etc/passwd # delimitador ': ', 3o campo numerico

# Ordenar por multiplos campos
$ sort -t: -k 3n -k 1,1 /etc/passwd

# Ordenar aleatoriamente (shuffle)
$ sort -R arquivo.txt

# Ordenar e verificar se ja esta ordenado
$ sort -c arquivo.txt

# Ordenar com locale especifico
$ LC_ALL=C sort arquivo.txt

# Ordenar arquivo grande (merge sort externo)
$ sort -S 50% arquivo_grande.txt
# Usa 50% da RAM

# Ordenar saida de comando
$ ps aux | sort -k 3 -n -r # por CPU%, numerico, reverso
$ du -h /home | sort -h -r # por tamanho, maior primeiro
```

3.6.1 Exemplos Praticos

```
# Top 10 processos por uso de memoria
$ ps aux | sort -k 4 -n -r | head -10

# Ordenar /etc/passwd por UID
$ sort -t: -k 3n /etc/passwd

# Ordenar arquivos por tamanho
$ ls -l | sort -k 5 -n

# Ordenar IPs numericamente
$ sort -t. -k1,1n -k2,2n -k3,3n -k4,4n ips.txt
```

3.7 uniq — Remove Duplicate Lines

O **uniq** remove ou conta linhas duplicadas adjacentes. **Importante:** so remove duplicatas adjacentes — use **sort | uniq** para remover todas.

```
# Remover duplicatas adjacentes
$ uniq arquivo.txt

# Contar ocorrencias
$ uniq -c arquivo.txt

# Mostrar apenas duplicatas
$ uniq -d arquivo.txt

# Mostrar apenas unicas (nao duplicadas)
$ uniq -u arquivo.txt
```

```
# Ignorar primeiros N caracteres
$ uniq -s 5 arquivo.txt

# Comparar apenas primeiros N caracteres
$ uniq -w 10 arquivo.txt

# Ignorar case
$ uniq -i arquivo.txt

# Combinar com sort (remover TODAS as duplicatas)
$ sort arquivo.txt | uniq
# ou mais eficiente:
$ sort -u arquivo.txt

# Contar ocorrencias de cada linha (histograma)
$ sort arquivo.txt | uniq -c | sort -n -r

# Mostrar duplicatas com contexto
$ uniq -D arquivo.txt
```

3.8 wc — Word Count

O **wc** conta linhas, palavras e bytes em arquivos ou entrada padrao.

```
# Contar linhas, palavras e bytes
$ wc arquivo.txt
  10   50  300 arquivo.txt
# 10 linhas, 50 palavras, 300 bytes

# Apenas linhas
$ wc -l arquivo.txt

# Apenas palavras
$ wc -w arquivo.txt

# Apenas bytes
$ wc -c arquivo.txt

# Apenas caracteres (multibyte)
$ wc -m arquivo.txt

# Maior largura de linha
$ wc -L arquivo.txt

# Contar linhas de multiplos arquivos
$ wc -l *.txt

# Contar linhas de saida de comando
$ ps aux | wc -l
$ find /home -type f | wc -l

# Contar arquivos em diretorio
$ ls -l | wc -l

# Contar palavras em string
$ echo "Hello World" | wc -w
```

3.9 cut — Remove Sections from Lines

O **cut** extrai colunas/bytes de cada linha. Util para processar arquivos delimitados.

```
# Extrair por campo (delimitador padrao: tab)
$ cut -f 1 arquivo.txt      # 1o campo
$ cut -f 1,3 arquivo.txt    # 1o e 3o campos
$ cut -f 1-3 arquivo.txt    # campos 1 a 3

# Extrair com delimitador customizado
$ cut -d: -f 1 /etc/passwd  # usuarios
$ cut -d: -f 1,3 /etc/passwd # usuario e UID
$ cut -d: -f 6 /etc/passwd  # home directory

# Extrair por bytes
$ cut -b 1-10 arquivo.txt

# Extrair por caracteres
$ cut -c 1-10 arquivo.txt

# Extrair complemento (tudo EXCETO)
$ cut --complement -d: -f 2 /etc/passwd
# Todos os campos EXCETO o 2o

# Combinar com outros comandos
$ ps aux | cut -c 1-10      # primeiros 10 chars
$ df -h | cut -d' ' -f 1    # filesystems
```

3.9.1 Exemplos Praticos

```
# Listar apenas nomes de usuarios
$ cut -d: -f1 /etc/passwd

# Listar IPs de conexoes ativas
$ ss -tuln | cut -d: -f2 | cut -d' ' -f2

# Extrair 2o campo de CSV
$ cut -d, -f2 dados.csv

# Extrair PID de processos
$ ps aux | cut -c 10-15
```

3.10 paste — Merge Lines of Files

O **paste** combina linhas de multiplos arquivos lado a lado (horizontalmente).

```
# Combinar dois arquivos lado a lado
$ paste arquivo1.txt arquivo2.txt

# Combinar com delimitador customizado
$ paste -d, nomes.txt emails.txt > contatos.csv

# Combinar em formato de tabela
$ paste -d'\t' col1.txt col2.txt col3.txt
```

```
# Combinar multiplos arquivos
$ paste arquivo1.txt arquivo2.txt arquivo3.txt

# Combinar linhas de um unico arquivo (serial)
$ paste -s -d',' numeros.txt
# 1,2,3,4,5 (tudo em uma linha)

# Usar com process substitution
$ paste <(cut -d: -f1 /etc/passwd) <(cut -d: -f6 /etc/passwd)
```

3.11 tr — Translate or Delete Characters

O **tr** traduz, comprime ou deleta caracteres. Opera apenas em stdin (nao aceita arquivo como argumento).

```
# Converter minusculas para maiusculas
$ echo "hello" | tr 'a-z' 'A-Z'
HELLO

# Converter maiusculas para minusculas
$ echo "HELLO" | tr 'A-Z' 'a-z'
hello

# Deletar caracteres
$ echo "hello 123 world" | tr -d '0-9'
hello world

# Deletar espacos
$ echo "hello world" | tr -d ' '
helloworld

# Comprimir caracteres repetidos
$ echo "hello world" | tr -s ' '
hello world

# Substituir espacos por tabs
$ echo "col1 col2 col3" | tr ' ' '\t'

# Substituir newlines por espacos
$ cat arquivo.txt | tr '\n' ' '

# Remover caracteres nao imprimiveis
$ tr -cd '[:print:]\n' < arquivo.txt

# Usar classes de caracteres
$ echo "hello123" | tr '[:lower:]' '[:upper:]'
$ echo "hello 123" | tr -d '[:digit:]'
$ echo "hello!!!" | tr -d '[:punct:]'

# Rot13 (cifra simples)
$ echo "hello" | tr 'A-Za-z' 'N-ZA-Mn-za-m'
uryyb

# Criar senha aleatoria
$ tr -dc 'A-Za-z0-9' < /dev/urandom | head -c 16

# Contar caracteres unicos
$ echo "hello" | tr 'a-z' '\n' | sort | uniq -c
```

3.11.1 Classes de Caracteres do tr

Classe	Descricao
<code>[:alnum:]</code>	Alfanumericos
<code>[:alpha:]</code>	Letras
<code>[:digit:]</code>	Digitos
<code>[:lower:]</code>	Minusculas
<code>[:upper:]</code>	Maiusculas
<code>[:space:]</code>	Espacos
<code>[:punct:]</code>	Pontuacao
<code>[:print:]</code>	Imprimiveis
<code>[:cntrl:]</code>	Controle

3.12 sed — Stream Editor

O `sed` eh um editor de streams nao-interativo. Pode substituir, deletar, inserir, e transformar texto linha por linha.

3.12.1 Substituicao (s///)

```
# Substituir primeira ocorrencia por linha
$ sed 's/erro/ERRO/' log.txt

# Substituir TODAS as ocorrencias (global)
$ sed 's/erro/ERRO/g' log.txt

# Substituir apenas No ocorrencia
$ sed 's/erro/ERRO/2' log.txt # 2a ocorrencia
$ sed 's/erro/ERRO/2g' log.txt # da 2a em diante

# Substituir com regex
$ sed 's/erro[0-9]*/ERRO/g' log.txt

# Usar delimitador diferente (uteis para paths)
$ sed 's|/home/usuario|/home/novo|g' arquivo.txt
$ sed 's#/usr/local#/opt#g' arquivo.txt

# Substituir e salvar no mesmo arquivo (in-place)
$ sed -i 's/erro/ERRO/g' log.txt

# Backup antes de editar
$ sed -i.bak 's/erro/ERRO/g' log.txt
# Cria log.txt.bak

# Substituir com referencia ao match
$ sed 's/(erro\)/[\1]/g' log.txt
```

```
# [erro]

# Substituir com grupos numerados
$ echo "2024-06-14" | sed 's/\([0-9]*\)-\([0-9]*\)-\([0-9]*\)/\3\/\2\/\1/'
14/06/2024
```

3.12.2 Deletar Linhas

```
# Deletar linha 5
$ sed '5d' arquivo.txt

# Deletar linhas 5 a 10
$ sed '5,10d' arquivo.txt

# Deletar linhas que contem padrao
$ sed '/erro/d' log.txt

# Deletar linhas que NAO contem padrao
$ sed '/erro/!d' log.txt

# Deletar linhas em branco
$ sed '/^$/d' arquivo.txt

# Deletar linhas comentadas
$ sed '/^#/d' arquivo.txt

# Deletar primeira e ultima linha
$ sed '1d;$d' arquivo.txt

# Deletar a ultima linha
$ sed '$d' arquivo.txt
```

3.12.3 Inserir e Adicionar

```
# Inserir antes da linha 1
$ sed '1i\# Header' arquivo.txt

# Adicionar depois da linha 1
$ sed '1a\nova linha' arquivo.txt

# Inserir antes de linhas com padrao
$ sed '/erro/i\--- ERRO ENCONTRADO ---' log.txt

# Adicionar ao final do arquivo
$ sed '$a\Fim do arquivo' arquivo.txt

# Substituir linha inteira
$ sed '5c\nova linha 5' arquivo.txt
```

3.12.4 sed Avancado

```
# Imprimir apenas linhas com padrao
$ sed -n '/erro/p' log.txt

# Equivalente a: grep "erro" log.txt

# Imprimir linhas 5 a 10
```

```

$ sed -n '5,10p' arquivo.txt

# Imprimir ultima linha
$ sed -n '$p' arquivo.txt

# Ler arquivo e inserir conteudo
$ sed '3r outro_arquivo.txt' arquivo.txt

# Executar multiplos comandos
$ sed -e 's/erro/ERRO/g' -e 's/falha/FALHA/g' log.txt

# Script sed em arquivo
$ sed -f script.sed arquivo.txt

# Usar expressoes estendidas
$ sed -E 's/([0-9]+)/[\1]/g' arquivo.txt

# Substituir newline por espaco (modo multilinha)
$ sed ':a;N;$!ba;s/\n/ /g' arquivo.txt

# Remover espacos no inicio das linhas
$ sed 's/^[ \t]*// ' arquivo.txt

# Remover espacos no final das linhas
$ sed 's/[ \t]*$// ' arquivo.txt

# Remover espacos no inicio e fim
$ sed 's/^[ \t]*//;s/[ \t]*$// ' arquivo.txt

# Duplicar cada linha
$ sed 'p' arquivo.txt

# Inverter linhas (tac com sed)
$ sed '1!G;h;$!d' arquivo.txt

```

sed -i eh PERIGOSO

`sed -i` modifica o arquivo original. Sempre teste sem `-i` primeiro, ou use `sed -i.bak` para criar backup. Para processamento critico, redirecione para novo arquivo: `sed 's/.../.../' arquivo.txt > arquivo_novo.txt`

3.13 awk — Pattern Scanning and Processing

O `awk` eh uma linguagem de programacao completa para processamento de texto. Pode fazer calculos, condicionais, loops, e manipular dados estruturados.

3.13.1 Uso Basico

```

# Imprimir campo especifico (delimitador padrao: espaco/tab)
$ awk '{print $1}' arquivo.txt

# Imprimir multiplos campos
$ awk '{print $1, $3}' arquivo.txt

# Usar delimitador customizado

```

```
$ awk -F: '{print $1}' /etc/passwd

# Imprimir com formato
$ awk '{print "Usuario:", $1, "- UID:", $3}' /etc/passwd

# Imprimir numero de campos
$ awk '{print NF}' arquivo.txt

# Imprimir linha inteira
$ awk '{print $0}' arquivo.txt

# Imprimir ultimo campo
$ awk '{print $NF}' arquivo.txt

# Imprimir com separador de saida
$ awk -F: -v OFS=' | ' '{print $1, $6}' /etc/passwd
```

3.13.2 Padroes e Condicoes

```
# Filtrar linhas com padrao
$ awk '/erro/' log.txt

# Filtrar por valor de campo
$ awk '$3 > 100' arquivo.txt

# Filtrar com regex no campo
$ awk '$2 ~ /^[0-9]/' arquivo.txt

# Multiplas condicoes
$ awk '$3 > 100 && $5 == "ativo"' arquivo.txt

# Usar BEGIN e END
$ awk 'BEGIN {print "Inicio"} {print $0} END {print "Fim"}' arquivo.txt

# Contar linhas
$ awk 'END {print NR}' arquivo.txt

# Somar valores de campo
$ awk '{soma += $3} END {print "Total:", soma}' arquivo.txt

# Media
$ awk '{soma += $3; count++} END {print "Media:", soma/count}' arquivo.txt

# Maximo e minimo
$ awk 'NR==1 {max=$3; min=$3} $3>max {max=$3} $3<min {min=$3} END {print "Max:", max, "Min:",
```

3.13.3 awk Avancado

```
# Usar variaveis
$ awk -v limite=100 '$3 > limite' arquivo.txt

# Usar arrays
$ awk '{count[$1]++} END {for (u in count) print u, count[u]}' log.txt

# Ordenar saida (pipe para sort)
$ awk '{count[$1]++} END {for (u in count) print count[u], u}' log.txt | sort -n -r

# Formatar saida com printf
```



```
$ awk '{printf "%-15s %10d\n", $1, $3}' arquivo.txt

# Processar CSV
$ awk -F, '{print $2}' dados.csv

# Substituir dentro do awk
$ awk '{gsub(/erro/, "ERRO"); print}' log.txt

# Substr (extrair parte do campo)
$ awk '{print substr($1, 1, 3)}' arquivo.txt

# Length (tamanho da linha/campo)
$ awk '{print length($0)}' arquivo.txt

# Split (dividir campo)
$ awk '{split($1, a, "."); print a[1]}' arquivo.txt

# Match (encontrar posicao do regex)
$ awk '{if (match($0, /[0-9]+/)) print substr($0, RSTART, RLENGTH)}' arquivo.txt

# Ler multiplos arquivos
$ awk '{print FILENAME, $0}' arquivo1.txt arquivo2.txt

# Processar registros multilinha
$ awk 'BEGIN {RS=""; FS="\n"} {print $1}' arquivo.txt
```

3.13.4 Exemplos Praticos de awk

```
# Top 5 processos por memoria
$ ps aux | awk '{print $4, $11}' | sort -n -r | head -5

# Soma de uso de disco por usuario
$ du -sh /home/* | awk '{sum += $1} END {print sum}'

# Extrair IPs de log
$ awk '{print $1}' /var/log/apache2/access.log | sort | uniq -c | sort -n -r

# Contar status HTTP
$ awk '{print $9}' /var/log/apache2/access.log | sort | uniq -c

# Relatorio de tamanho de diretorios
$ du -sh /home/* | awk '{printf "%-20s %10s\n", $2, $1}'

# Filtrar /etc/passwd: usuarios com UID >= 1000
$ awk -F: '$3 >= 1000 {print $1, $6}' /etc/passwd

# Calcular tempo total de processos
$ ps -eo time | awk -F: '{sum += $1*3600 + $2*60 + $3} END {print sum "s"}'
```

✓ Quando usar grep, sed ou awk?

Ferramenta	Quando usar
grep	Buscar padroes, filtrar linhas
sed	Substituir texto, deletar/inserir linhas

awk

Processar campos, cálculos, relatórios

Muitas vezes você os usará juntos: `grep` para filtrar, `sed` para limpar, `awk` para processar.

🔧 Exercícios — Volume 3

1. Use `grep` para encontrar todas as linhas que começam com "#" em `/etc/services`. Conte quantas são.
2. Use `grep -E` para buscar linhas que contenham "tcp" OU "udp" em `/etc/services`.
3. Use `find` para listar todos os arquivos `.conf` em `/etc` maiores que 1KB. Ordene por tamanho.
4. Use `find` para encontrar arquivos modificados nas últimas 24h no seu home. Execute `ls -lh` para cada um.
5. Use `sort` e `uniq -c` para criar um histograma das shells usadas em `/etc/passwd` (campo 7).
6. Use `cut` para extrair apenas os nomes de usuário e diretórios home de `/etc/passwd`, formatados como "usuário → home".
7. Use `tr` para converter o conteúdo de um arquivo para maiúsculas e contar quantos caracteres são dígitos.
8. Use `sed` para remover TODOS os comentários (linhas começando com #) de um arquivo de configuração.
9. Use `sed` para substituir todas as ocorrências de "localhost" por "127.0.0.1" em um arquivo, salvando backup.
10. Use `awk` para criar um relatório de `ps aux` mostrando: usuário, %CPU, %MEM, comando — ordenado por %CPU.
11. Use `awk` para processar `/etc/passwd` e mostrar apenas usuários com UID >= 1000, formatados em tabela.
12. Crie uma pipeline que: lista arquivos `.log` em `/var/log`, filtra os que contêm "error", extrai a data, e conta ocorrências por dia.

Volume 4 — Pipes e Redirecionamentos

A alma do Unix: composicao de programas simples para criar solucoes poderosas.

4.1 Os Tres Fluxos de Dados

Todo processo no Linux tem tres fluxos de dados padrao:

```
PROCESSO | | | stdin (0) —→ programa | | ↑ | |
teclado / pipe / arquivo | | | stdout (1) —→ tela / pipe | | ↓ | | terminal / arquivo | | |
| stderr (2) —→ tela / arquivo | | ↓ | | terminal / arquivo |
```

Fluxo	Descritor	Descricao	Uso Padrao
stdin	0	Entrada padrao	Teclado
stdout	1	Saida padrao	Terminal
stderr	2	Saida de erro	Terminal

Esses fluxos sao implementados como **arquivos** no Linux. O kernel abre tres descritores de arquivo (file descriptors) para cada processo: 0 para stdin, 1 para stdout, e 2 para stderr.

4.2 Redirecionamento de Saida (stdout) — > e >>

4.2.1 Sobrescrever Arquivo (>)

```
# Salvar saida em arquivo (SOBRESCREVE se existir)
$ echo "Hello" > arquivo.txt

# Salvar saida de comando
$ ls -la > lista.txt

# Salvar saida de programa
$ ./programa > saida.log

# Redirecionar para /dev/null (descartar)
$ ./programa > /dev/null
```

> SOBRESCREVE sem aviso!

Se `arquivo.txt` ja existir, `>` o apaga e cria um novo. Nao ha lixeira. Use `>>` para anexar, ou `set -o noclobber` no bash para prevenir sobrescrita acidental.

4.2.2 Anexar a Arquivo (>>)

```
# Anexar saida ao final do arquivo
$ echo "Nova linha" >> arquivo.txt

# Acumular logs
$ ./programa >> log.txt

# Adicionar ao .bashrc
$ echo 'alias ll="ls -la"' >> ~/.bashrc
```

4.2.3 noclobber — Prevenir Sobreescrita

```
# Habilitar protecao
$ set -o noclobber

# Agora isso falha se arquivo existir
$ echo "teste" > arquivo_existente.txt
bash: arquivo_existente.txt: cannot overwrite existing file

# Forcar sobreescrita mesmo com noclobber
$ echo "teste" >| arquivo_existente.txt

# Desabilitar protecao
$ set +o noclobber
```

4.3 Redirecionamento de Entrada (stdin) — <

```
# Usar arquivo como entrada
$ sort < nomes.txt

# Contar palavras de arquivo
$ wc -w < texto.txt

# Usar heredoc como entrada
$ cat << EOF
Linha 1
Linha 2
EOF

# Usar here-string
$ tr 'a-z' 'A-Z' <<< "hello"
HELLO

# Redirecionar entrada e saida
$ sort < nomes.txt > nomes_ordenados.txt
```

4.3.1 Heredoc (<<)

```
# Heredoc basico
$ cat << EOF
Este eh um texto
com multiplas linhas
EOF
```

```
# Heredoc sem expansao de variaveis (aspas no delimitador)
$ cat << 'EOF'
$HOME nao sera expandido
EOF

# Heredoc com tab stripping (<<-)
$ cat <<- EOF
    Esta linha tera o tab removido
        Esta tambem
EOF

# Heredoc em script SQL
$ mysql -u root << 'SQL'
SELECT * FROM usuarios;
SQL
```

4.3.2 Here-string (<<<)

```
# Passar string como stdin
$ grep "padrao" <<< "texto com padrao aqui"

# Usar com comando que espera arquivo
$ base64 <<< "senha123"

# Processar variavel
$ texto="hello world"
$ tr 'a-z' 'A-Z' <<< "$texto"
```

4.4 Redirecionamento de Erro (stderr) — 2> e 2>>

```
# Redirecionar apenas erros para arquivo
$ ./programa 2> erros.log

# Redirecionar erros e descartar
$ ./programa 2> /dev/null

# Acumular erros
$ ./programa 2>> erros.log

# Separar stdout e stderr
$ ./programa > saida.log 2> erros.log

# Juntar stdout e stderr no mesmo arquivo
$ ./programa > tudo.log 2>&1

# Ordem importa! Isso funciona:
$ ./programa > tudo.log 2>&1

# Isso NAO funciona como esperado (stderr vai para tela):
$ ./programa 2>&1 > tudo.log
```

4.4.1 Combinando stdout e stderr

```
# Ambos para o mesmo arquivo
```

```
$ comando > arquivo.log 2>&1

# Ambos para /dev/null
$ comando > /dev/null 2>&1
# Atalho em bash 4+: &> /dev/null

# stdout para arquivo, stderr para outro
$ comando > saida.log 2> erros.log

# stdout para pipe, stderr para arquivo
$ comando 2> erros.log | outro_comando

# stderr para stdout, depois pipe
$ comando 2>&1 | grep "erro"
```

Por que 2>&1 e nao 2>1?

2>1 redirecionaria stderr para um *arquivo* chamado "1". **2>&1** significa "redirecione stderr (2) para onde stdout (1) esta apontando". O **&** indica que o numero seguinte eh um descritor de arquivo, nao um nome de arquivo.

4.5 Pipe (|) — Conectar Comandos

O pipe conecta o **stdout** de um comando ao **stdin** de outro. E a característica mais poderosa do shell Unix.

```
# Pipeline basica
$ ls -la | grep ".txt"

# Multiplos pipes
$ cat log.txt | grep "erro" | sort | uniq -c | sort -n -r

# Contar processos
$ ps aux | wc -l

# Top 10 maiores diretorios
$ du -h /home | sort -h -r | head -10

# Filtrar e processar
$ cat /etc/passwd | grep "/bin/bash" | cut -d: -f1

# Pipeline com awk
$ ps aux | awk '{print $2, $11}' | grep "firefox"

# Pipeline com sed
$ ifconfig | grep "inet " | sed 's/inet //' | awk '{print $1}'
```

4.5.1 Pipe com stderr

```
# Pipe apenas stdout
$ comando | grep "padrao"

# Pipe stdout e stderr juntos
$ comando 2>&1 | grep "padrao"
```

```
# Pipe apenas stderr (troca stdout e stderr)
$ comando 3>&1 1>&2 2>&3 | grep "erro"
```

4.5.2 Process Substitution (<())

```
# Comparar saída de dois comandos
$ diff <(ls dir1) <(ls dir2)

# Usar saída de comando como arquivo
$ grep "padrao" <(cat arquivo1.txt arquivo2.txt)

# Combinar múltiplas saídas
$ cat <(echo "Header") <(cat dados.txt) <(echo "Footer")

# Process substitution com escrita
$ tee >(gzip > backup.gz) < arquivo.txt
```

4.6 tee — Lê stdin e Escreve em stdout e Arquivos

O **tee** resolve o problema de querer ver a saída na tela E salvar em arquivo ao mesmo tempo.

```
# Ver na tela e salvar em arquivo
$ ls -la | tee lista.txt

# Anexar em vez de sobrescrever
$ echo "nova linha" | tee -a log.txt

# Escrever em múltiplos arquivos
$ comando | tee arquivo1.txt arquivo2.txt

# Usar com sudo (permite redirecionamento com privilégios)
$ echo "config" | sudo tee /etc/arquivo.conf

# Pipe para tee e depois para outro comando
$ comando | tee log.txt | grep "erro"

# Tee com process substitution
$ comando | tee >(grep "erro" > erros.log) >(grep "aviso" > avisos.log)

# Tee para /dev/null (apenas log)
$ comando | tee /dev/tty | gzip > saida.gz
```

```
comando | ▼ stdout ┌───┐ | tee |───┐ stdout (tela) ┌───┐ | ▼ arquivo.txt
```

4.7 Redirecionamentos Avançados

4.7.1 Redirecionar Múltiplos Descritores

```
# Fechar stdin
$ comando <&-
```

```
# Fechar stdout
$ comando >&-

# Duplicar descritores
$ comando 3>&1 # fd 3 aponta para stdout
$ comando 1>&3 # stdout aponta para fd 3

# Trocar stdout e stderr
$ comando 3>&1 1>&2 2>&3

# Redirecionar stdin e stdout de/para o mesmo arquivo
$ sed -i 's/foo/bar/' arquivo.txt
# (sed -i faz isso internamente)
```

4.7.2 Here-document em Scripts

```
#!/bin/bash

# Criar arquivo de configuracao
config=$(cat <<- 'CFG'
[database]
host=localhost
port=3306
user=admin
pass=secret

[server]
port=8080
debug=false
CFG
)

echo "$config" > config.ini

# Executar SQL
mysql -u root <<- 'SQL'
CREATE DATABASE IF NOT EXISTS meu_db;
USE meu_db;
CREATE TABLE usuarios (id INT, nome VARCHAR(100));
SQL
```

4.7.3 /dev/null e /dev/zero

```
# Descartar toda saida
$ comando > /dev/null 2>&1

# Fornecer zeros como entrada
$ dd if=/dev/zero of=arquivo.img bs=1M count=100

# Fornecer dados aleatorios
$ head -c 100 /dev/urandom > aleatorio.bin

# Fornecer dados aleatorios criptograficamente seguros
$ head -c 100 /dev/random > seguro.bin
# /dev/random pode bloquear esperando entropia
```

4.7.4 Named Pipes (FIFOs)


```
# Criar named pipe
$ mkfifo meu_pipe

# Em um terminal: escrever no pipe
$ echo "dados" > meu_pipe

# Em outro terminal: ler do pipe
$ cat meu_pipe
dados

# Usar em pipeline complexo
$ mkfifo pipe1 pipe2
$ comando1 > pipe1 &
$ comando2 < pipe1 > pipe2 &
$ comando3 < pipe2

# Remover pipe
$ rm meu_pipe
```

✓ Resumo de Redirecionamentos

Redirecionamento	Significado
<code>cmd > file</code>	stdout para arquivo (sobrescreve)
<code>cmd >> file</code>	stdout para arquivo (anexa)
<code>cmd 2> file</code>	stderr para arquivo
<code>cmd &> file</code>	stdout e stderr para arquivo
<code>cmd > file 2>&1</code>	stdout e stderr para arquivo
<code>cmd < file</code>	stdin de arquivo
<code>cmd << EOF</code>	stdin de heredoc
<code>cmd <<< string</code>	stdin de here-string
<code>cmd1 cmd2</code>	stdout de cmd1 para stdin de cmd2
<code>cmd tee file</code>	stdout para tela E arquivo
<code>cmd > /dev/null</code>	Descarta stdout

🔧 Exercícios — Volume 4

1. Execute `ls /nao_existe > saida.txt 2> erro.txt`. Verifique o conteúdo de cada arquivo. O que acontece?
2. Execute `ls /nao_existe > tudo.txt 2>&1`. Verifique o arquivo. Por que a ordem `2>&1` importa?

3. Crie uma pipeline que: lista todos os processos, filtra os do seu usuário, ordena por %CPU, e mostra os top 5.
4. Use `tee` para executar `ps aux`, salvar em `processos.txt`, e filtrar apenas "firefox" na tela.
5. Crie um heredoc que gere um arquivo HTML completo com `cat > pagina.html`.
6. Use process substitution para comparar `ls /bin` e `ls /usr/bin` com `diff`.
7. Crie um script que execute um comando, salve stdout em `ok.log` e stderr em `erro.log`, separadamente.
8. Use `find` com `-exec` e redirecionamento para criar um arquivo com a lista de todos os `.conf` em `/etc`.
9. Crie um named pipe e use-o para passar dados entre dois terminais.
10. Escreva uma pipeline de 5+ comandos que processe `/var/log/syslog` e gere um relatório de erros por hora.



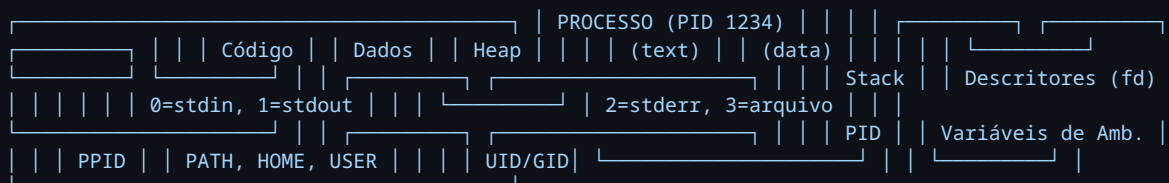
Volume 5 — Gerenciamento de Processos

Entender, monitorar e controlar processos é essencial para dominar Linux.

5.1 O que é um Processo?

Um **processo** é uma instância em execução de um programa. Cada processo tem:

- **PID** (Process ID): identificador único
- **PPID** (Parent PID): PID do processo pai
- **UID/GID**: usuário e grupo dono
- **Espaço de memória**: código, dados, heap, stack
- **Descritores de arquivo**: stdin (0), stdout (1), stderr (2), e outros
- **Variáveis de ambiente**
- **Diretório de trabalho**
- **Estado**: running, sleeping, stopped, zombie



5.2 ps — Report Process Status

O **ps** exibe informações sobre processos ativos. É um snapshot do momento da execução.

5.2.1 Uso Básico

```
# Processos do terminal atual
$ ps
  PID TTY          TIME CMD
 1234 pts/0    00:00:00 bash
 5678 pts/0    00:00:00 ps

# Todos os processos do usuário
$ ps -u $(whoami)

# Todos os processos (estilo BSD)
$ ps aux

# Todos os processos (estilo UNIX)
$ ps -ef

# Processos em formato customizado
$ ps -eo pid,ppid,cmd,%mem,%cpu
```

```
# Processos de um usuário específico
$ ps -u root

# Processos em uma árvore
$ ps axjf
```

5.2.2 Colunas do ps aux

Coluna	Significado
USER	Usuário dono do processo
PID	Process ID
%CPU	Porcentagem de uso de CPU
%MEM	Porcentagem de uso de memória
VSZ	Virtual Memory Size (KB)
RSS	Resident Set Size (memória física, KB)
TTY	Terminal associado
STAT	Estado do processo
START	Hora de início
TIME	Tempo total de CPU usado
COMMAND	Comando executado

5.2.3 Estados de Processo (STAT)

Código	Significado
R	Running (executando ou na fila)
S	Sleeping (interrompível, esperando evento)
D	Disk Sleep (não interrompível, geralmente I/O)
T	Stopped (pausado por sinal)
Z	Zombie (terminado, aguardando reaping pelo pai)
I	Idle (kernel thread inativa)

Modificadores:

Código	Significado

<	Alta prioridade (nice negativo)
N	Baixa prioridade (nice positivo)
+	Processo em foreground
l	Multi-threaded
s	Session leader

5.2.4 ps Avançado

```
# Formato customizado com todas as colunas
$ ps -eo pid,ppid,uid,gid,tty,stat,priority,nice,pcpu,pmem,comm,args

# Processos ordenados por uso de CPU
$ ps aux --sort=-%cpu | head -10

# Processos ordenados por uso de memória
$ ps aux --sort=-%mem | head -10

# Processos com threads
$ ps -eLf | head -10

# Mostrar árvore de processos
$ ps axjf

# Mostrar apenas PIDs
$ ps -eo pid=

# Contar processos
$ ps aux | wc -l

# Encontrar processo por nome
$ ps aux | grep firefox
$ pgrep firefox

# Informações detalhadas de um processo
$ ps -p 1234 -o pid,ppid,cmd,etime,pcpu,pmem
```

5.3 top — Display Linux Processes

O **top** é um monitor de processos interativo em tempo real.

```
# Iniciar top
$ top

# Iniciar com delay de 1 segundo
$ top -d 1

# Mostrar apenas processos de um usuário
$ top -u usuario

# Mostrar processos de um PID específico
$ top -p 1234,5678
```

```
# Executar N ciclos e sair
$ top -n 5

# Salvar saída em batch mode
$ top -b -n 1 > processos.txt
```

5.3.1 Comandos Interativos no top

Tecla	Ação
h ou ?	Ajuda
q	Sair
Space	Atualizar imediatamente
k	Matar processo (kill)
r	Renice (mudar prioridade)
u	Filtrar por usuário
M	Ordenar por memória
P	Ordenar por CPU
T	Ordenar por tempo
c	Mostrar caminho completo do comando
l	Mostrar CPUs individuais
d	Mudar delay de atualização
w	Salvar configuração

5.4 htop — Interactive Process Viewer

O **htop** é uma alternativa mais amigável ao **top**, com interface colorida e navegação por teclas de seta.

```
# Instalar htop
$ sudo apt install htop

# Iniciar htop
$ htop

# Filtrar por nome
$ htop -p 1234

# Mostrar árvore
$ htop -t

# Ordenar por coluna específica
# Use F6 para selecionar a coluna de ordenação
```

5.4.1 Atalhos do htop

Tecla	Ação
F1 ou h	Ajuda
F2 ou S	Setup (configuração)
F3 ou /	Buscar processo
F4 ou \	Filtrar
F5 ou t	Modo árvore
F6 ou >	Ordenar por coluna
F9 ou k	Matar processo
F10 ou q	Sair
Space	Selecionar/desmarcar processo
u	Filtrar por usuário

5.5 pstree — Display Process Tree

O **pstree** mostra processos em formato de árvore, facilitando a visualização da hierarquia.

```
# Árvore de todos os processos
$ pstree

# Árvore com PIDs
$ pstree -p

# Árvore de um usuário específico
$ pstree usuario

# Árvore de um PID específico
$ pstree 1

# Mostrar argumentos completos
$ pstree -a

# Mostrar threads
$ pstree -t

# Mostrar PGIDs
$ pstree -g

# Destacar ancestral do processo atual
$ pstree -h

# Combinar opções
$ pstree -pahlu
```

```
systemd(1)└─accounts-daemon(567)└─cron(589)└─dbus-daemon(576)
└─sshd(612)──sshd(1234)──bash(1235)──pstree(5678)└─systemd-journal(345)└─systemd-logind(456)
```

5.6 Jobs, Foreground e Background

O shell pode gerenciar múltiplos processos em uma sessão. Processos podem rodar em **foreground** (interativo) ou **background** (não-interativo).

5.6.1 Controle de Jobs

```
# Executar em background
$ sleep 60 &
[1] 1234

# Listar jobs
$ jobs
[1]+  Running                  sleep 60 &

# Trazer para foreground
$ fg %1

# Mandar para background
# Ctrl+Z para suspender, depois:
$ bg %1

# Suspender processo em foreground
# Ctrl+Z

# Matar um job
$ kill %1

# Trazer último job para foreground
$ fg

# Trazer job específico
$ fg %2

# Trazer job pelo nome
$ fg %sleep
```

5.6.2 Estados de Jobs

Estado	Significado
Running	Executando em background
Stopped	Suspenso (Ctrl+Z)
Done	Terminado com sucesso
Done (code)	Terminado com código de saída
Terminated	Terminado por sinal

5.6.3 nohup — Run Command Immune to Hangups

```
# Executar comando que sobrevive ao logout
$ nohup ./script.sh &
[1] 1234
nohup: ignoring input and appending output to 'nohup.out'

# Especificar arquivo de saída
$ nohup ./script.sh > meu_log.txt 2>&1 &

# Combinar com disown
$ ./script.sh &
$ disown %1
# O processo continua mesmo se o terminal fechar
```

nohup vs. disown vs. screen/tmux

- **nohup** : Executa comando ignorando SIGHUP. Simples, mas sem interação.
- **disown** : Remove job da tabela de jobs do shell. O processo continua, mas sem controle fácil.
- **screen** / **tmux** : Emuladores de terminal persistentes. Permitem reconectar posteriormente. Recomendado para sessões longas.

5.7 kill, killall, pkill — Enviar Sinais

Processos no Linux se comunicam via **sinais**. O comando **kill** envia sinais para processos.

5.7.1 Sinais Importantes

Sinal	Número	Ação	Uso
SIGHUP	1	Hang up	Recarregar config (daemon)
SIGINT	2	Interrupt	Ctrl+C
SIGQUIT	3	Quit	Ctrl+\, gera core dump
SIGKILL	9	Kill	Matar imediatamente (não capturável)
SIGTERM	15	Terminate	Terminar graciosamente (padrão)
SIGSTOP	19	Stop	Ctrl+Z, não capturável
SIGCONT	18	Continue	Continuar processo parado
SIGUSR1	10	User 1	Uso definido pelo programa
SIGUSR2	12	User 2	Uso definido pelo programa

5.7.2 Uso do kill

```
# Enviar SIGTERM (padrão) para um PID
$ kill 1234

# Enviar SIGKILL (forçado)
$ kill -9 1234
$ kill -SIGKILL 1234

# Enviar SIGHUP (recarregar config)
$ kill -HUP 1234
$ kill -1 1234

# Enviar SIGINT (equivalente a Ctrl+C)
$ kill -INT 1234

# Enviar SIGSTOP (pausar)
$ kill -STOP 1234

# Enviar SIGCONT (continuar)
$ kill -CONT 1234

# Listar todos os sinais disponíveis
$ kill -l

# Verificar se processo existe
$ kill -0 1234
# Código 0 = processo existe; código != 0 = não existe
```

5.7.3 killall — Kill by Name

```
# Matar todos os processos com nome
$ killall firefox

# Matar com sinal específico
$ killall -9 firefox

# Listar processos que seriam mortos (simulação)
$ killall -i firefox

# Matar por usuário
$ sudo killall -u usuario firefox

# Matar processos mais antigos que N segundos
$ killall -o 1h firefox

# Matar processos mais novos que N segundos
$ killall -y 1m firefox

# Considerar nome exato
$ killall -e firefox
```

5.7.4 pkill — Kill by Pattern

```
# Matar por padrão no nome
$ pkill firefox

# Matar por padrão no comando completo
$ pkill -f "python script.py"
```

```
# Matar por usuário
$ pkill -u usuario

# Matar por terminal
$ pkill -t pts/0

# Enviar sinal específico
$ pkill -9 firefox

# Listar PIDs que seriam afetados
$ pgrep firefox

# Combinar critérios
$ pkill -u usuario -f "python"
```

⚠ **killall pode ser perigoso**

killall mata TODOS os processos com aquele nome. Em alguns sistemas Unix (Solaris, AIX), **killall** mata TODOS os processos do sistema! Sempre use **pgrep** primeiro para verificar quais processos seriam afetados.

5.8 nice e renice — Ajustar Prioridade

A **nice value** (-20 a 19) determina a prioridade de um processo. Valores mais baixos = maior prioridade.

```
# Iniciar processo com prioridade baixa (nice alto)
$ nice -n 10 ./script.sh

# Iniciar com prioridade alta (requer root)
$ sudo nice -n -10 ./script.sh

# Alterar prioridade de processo em execução
$ renice -n 5 -p 1234

# Alterar prioridade de todos os processos de um usuário
$ sudo renice -n 5 -u usuario

# Alterar prioridade por nome
$ sudo renice -n -5 $(pgrep firefox)

# Ver nice value
$ ps -eo pid,comm,nice | grep firefox
```

Nice Value	Prioridade	Uso
-20	Máxima	Processos críticos de tempo real
-10	Alta	Compilações, builds
0	Normal	Padrão
10	Baixa	Backups, indexação

🔧 Exercícios — Volume 5

1. Execute `ps aux` e identifique: seu shell, o processo init (systemd), e 3 processos do sistema.
2. Use `ps -eo pid,ppid,stat,cmd` e encontre um processo em estado `Z` (zombie). Se não houver, pesquise como criar um.
3. Abra `top`, pressione `1` para ver CPUs individuais, `M` para ordenar por memória, e identifique o processo que mais consome recursos.
4. Instale e use `htop`. Explore a interface, filtre por usuário, e mude a ordenação.
5. Use `pstree -p` para ver a árvore completa do sistema. Encontre o seu shell e trace até o PID 1.
6. Execute `sleep 300`, suspenda com Ctrl+Z, verifique com `jobs`, e continue em background com `bg`.
7. Use `nohup` para executar um comando longo, feche o terminal, e verifique se o processo ainda está rodando em outro terminal.
8. Use `kill -l` para listar todos os sinais. Envie SIGTERM e depois SIGKILL para um processo de teste. Qual a diferença?
9. Use `nice` para iniciar um processo com prioridade 10 e `renice` para mudar para 5.
10. Crie um script que monitore o uso de CPU de um processo específico a cada 2 segundos usando `ps`.

Volume 6 — Redes no Linux

Diagnosticar, configurar e entender redes é fundamental para sistemas e segurança.

6.1 ip — Show/Manipulate Routing, Devices, Policy Routing and Tunnels

O `ip` é o comando moderno para gerenciamento de rede no Linux, substituindo `ifconfig`, `route`, `arp`, e outros.

6.1.1 Interfaces de Rede

```
# Listar todas as interfaces
$ ip link show

# Listar interfaces com estado
$ ip link show up

# Ver detalhes de uma interface
$ ip link show eth0

# Habilitar interface
$ sudo ip link set eth0 up

# Desabilitar interface
$ sudo ip link set eth0 down

# Alterar MTU
$ sudo ip link set eth0 mtu 9000

# Alterar MAC address
$ sudo ip link set eth0 address 00:11:22:33:44:55

# Listar interfaces de forma resumida
$ ip -s link

# Estatísticas de interface
$ ip -s link show eth0
```

6.1.2 Endereços IP

```
# Listar endereços IP
$ ip addr show
$ ip a

# Listar endereços de uma interface
$ ip addr show eth0

# Adicionar endereço IP
$ sudo ip addr add 192.168.1.100/24 dev eth0

# Remover endereço IP
$ sudo ip addr del 192.168.1.100/24 dev eth0
```

```
# Limpar todos os endereços de uma interface
$ sudo ip addr flush dev eth0

# Mostrar apenas IPv4
$ ip -4 addr show

# Mostrar apenas IPv6
$ ip -6 addr show
```

6.1.3 Rotas

```
# Mostrar tabela de rotas
$ ip route show
$ ip r

# Adicionar rota
$ sudo ip route add 10.0.0.0/8 via 192.168.1.1

# Adicionar rota padrão
$ sudo ip route add default via 192.168.1.1

# Remover rota
$ sudo ip route del 10.0.0.0/8

# Mostrar rota para destino específico
$ ip route get 8.8.8.8

# Mostrar tabela de rotas com detalhes
$ ip route show table main
$ ip route show table local

# Mostrar cache de rotas
$ ip route show cache
```

6.1.4 Vizinhos (ARP)

```
# Mostrar tabela ARP
$ ip neigh show
$ ip n

# Adicionar entrada ARP estática
$ sudo ip neigh add 192.168.1.1 lladdr 00:11:22:33:44:55 dev eth0

# Remover entrada
$ sudo ip neigh del 192.168.1.1 dev eth0

# Limpar cache ARP
$ sudo ip neigh flush all
```

6.2 ping — Send ICMP ECHO_REQUEST

O **ping** testa conectividade com outro host usando pacotes ICMP.

```
# Ping básico
```

```
$ ping google.com

# Ping com contagem limitada
$ ping -c 4 google.com

# Ping com intervalo específico (segundos)
$ ping -i 2 google.com

# Ping flood (requer root, cuidado!)
$ sudo ping -f google.com

# Ping com tamanho de pacote
$ ping -s 1400 google.com

# Ping com timeout
$ ping -w 10 google.com

# Ping com TTL específico
$ ping -t 64 google.com

# Ping com timestamp
$ ping -D google.com

# Ping para IPv6
$ ping6 google.com
$ ping -6 google.com

# Ping para broadcast
$ ping -b 192.168.1.255
```

6.2.1 Interpretando Resultados do ping

```
$ ping -c 4 google.com
PING google.com (142.250.80.46) 56(84) bytes of data.
64 bytes from ...: icmp_seq=1 ttl=117 time=15.3 ms
64 bytes from ...: icmp_seq=2 ttl=117 time=14.8 ms
64 bytes from ...: icmp_seq=3 ttl=117 time=15.1 ms
64 bytes from ...: icmp_seq=4 ttl=117 time=14.9 ms

--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 14.812/15.050/15.312/0.183 ms
```

Campo	Significado
<code>icmp_seq</code>	Sequência do pacote ICMP
<code>ttl</code>	Time To Live (saltos restantes)
<code>time</code>	Round-trip time (RTT)
<code>packet loss</code>	Porcentagem de pacotes perdidos

6.3 ss — Investigate Sockets

O **ss** é o substituto moderno do **netstat**, mais rápido e informativo.

```
# Listar todas as conexões
$ ss

# Listar todas as sockets (TCP, UDP, UNIX)
$ ss -a

# Listar apenas sockets TCP
$ ss -t

# Listar apenas sockets UDP
$ ss -u

# Listar sockets em estado LISTEN
$ ss -l

# Listar sockets com processos
$ ss -tp

# Listar sockets com numeração (não resolve nomes)
$ ss -n

# Combinar opções
$ ss -tlnp # TCP, listening, numeric, processes
$ ss -tunap # TCP+UDP, all, numeric, processes

# Filtrar por estado
$ ss state established
$ ss state listening
$ ss state time-wait

# Filtrar por porta
$ ss -tlnp sport = :80
$ ss -tlnp sport = :ssh

# Filtrar por endereço
$ ss -tlnp src 192.168.1.1

# Mostrar estatísticas
$ ss -s

# Mostrar sockets UNIX
$ ss -x

# Mostrar sockets raw
$ ss -w

# Mostrar socket memory usage
$ ss -m
```

6.3.1 Estados de Conexão TCP

Estado	Significado
ESTAB	Conexão estabelecida
SYN-SENT	Enviou SYN, aguardando SYN-ACK

SYN-RCVD	Recebeu SYN, enviou SYN-ACK
FIN-WAIT-1	Enviou FIN, aguardando ACK
FIN-WAIT-2	Recebeu ACK do FIN, aguardando FIN do peer
TIME-WAIT	Aguardando 2xMSL antes de fechar
CLOSE-WAIT	Recebeu FIN, aguardando aplicação fechar
LAST-ACK	Enviou FIN, aguardando ACK final
LISTEN	Aguardando conexões
CLOSING	Simultaneous close

6.4 netstat — Network Statistics

O `netstat` é o comando legado para estatísticas de rede. Ainda útil, mas `ss` é preferido.

```
# Listar todas as conexões
$ netstat -a

# Listar conexões TCP
$ netstat -at

# Listar conexões UDP
$ netstat -au

# Listar sockets em listening
$ netstat -l

# Listar com processos
$ sudo netstat -tlnp

# Listar com nomes resolvidos
$ netstat -tln

# Estatísticas de protocolo
$ netstat -s

# Tabela de rotas
$ netstat -r
$ netstat -rn

# Estatísticas de interface
$ netstat -i

# Conexões ativas com timers
$ netstat -o

# Mostrar multicast
$ netstat -g
```

6.5 curl — Transfer Data from URLs

O **curl** é uma ferramenta poderosa para transferir dados com URLs. Suporta HTTP, HTTPS, FTP, SFTP, e muitos outros protocolos.

```
# GET simples
$ curl https://api.example.com/dados

# Salvar em arquivo
$ curl -o arquivo.json https://api.example.com/dados

# Salvar com nome do servidor
$ curl -O https://example.com/arquivo.zip

# Seguir redirects
$ curl -L https://bit.ly/xyz

# Mostrar headers
$ curl -I https://example.com

# Mostrar headers e corpo
$ curl -i https://example.com

# Headers customizados
$ curl -H "Authorization: Bearer token123" https://api.example.com

# POST com dados
$ curl -X POST -d "nome=Joao&idade=25" https://api.example.com/usuarios

# POST com JSON
$ curl -X POST \
  -H "Content-Type: application/json" \
  -d '{"nome":"Joao","idade":25}' \
  https://api.example.com/usuarios

# POST com arquivo
$ curl -X POST -d @dados.json https://api.example.com/usuarios

# PUT
$ curl -X PUT -d @atualizacao.json https://api.example.com/usuarios/1

# DELETE
$ curl -X DELETE https://api.example.com/usuarios/1

# Autenticação básica
$ curl -u usuario:senha https://api.example.com

# Autenticação com arquivo .netrc
$ curl --netrc https://api.example.com

# Ignorar certificado SSL (INSEGURO!)
$ curl -k https://self-signed.example.com

# Usar certificado específico
$ curl --cacert ca.crt https://example.com

# Verbose (debug)
$ curl -v https://example.com

# Verbose com headers completos
$ curl -vv https://example.com

# Silent (sem progresso)
```

```

$ curl -s https://example.com

# Silent mas mostra erros
$ curl -sS https://example.com

# Mostrar apenas código HTTP
$ curl -s -o /dev/null -w "%{http_code}" https://example.com

# Download com resume
$ curl -C - -o arquivo.zip https://example.com/arquivo.zip

# Limitar velocidade de download
$ curl --limit-rate 100K -o arquivo.zip https://example.com/arquivo.zip

# Timeout
$ curl --connect-timeout 10 --max-time 30 https://example.com

# Download paralelo (múltiplas conexões)
$ curl -Z -o arquivo.zip https://example.com/arquivo.zip

# FTP
$ curl ftp://ftp.example.com/arquivo.txt
$ curl -u usuario:senha ftp://ftp.example.com/arquivo.txt

# Upload FTP
$ curl -T arquivo.txt ftp://ftp.example.com/

# SFTP
$ curl -u usuario sftp://servidor.com/arquivo.txt

# SCP
$ curl -u usuario scp://servidor.com/arquivo.txt

```

6.5.1 curl Avançado

```

# Cookies
$ curl -c cookies.txt https://example.com/login
$ curl -b cookies.txt https://example.com/perfil

# Proxy
$ curl -x http://proxy:8080 https://example.com
$ curl -x socks5://proxy:1080 https://example.com

# User-Agent customizado
$ curl -A "Mozilla/5.0" https://example.com

# Referer
$ curl -e "https://google.com" https://example.com

# Enviar arquivo multipart/form-data
$ curl -F "arquivo=@foto.jpg" https://example.com/upload
$ curl -F "nome=Joao" -F "arquivo=@foto.jpg" https://example.com/upload

# WebSocket (com upgrade)
$ curl -i -N \
  -H "Connection: Upgrade" \
  -H "Upgrade: websocket" \
  https://example.com/ws

# Formatar JSON de saída (com jq)

```

```
$ curl -s https://api.example.com/dados | jq
```

6.6 wget — The Non-Interactive Network Downloader

O **wget** é especializado em downloads. Diferente de **curl**, ele é não-interativo e ideal para scripts de download.

```
# Download simples
$ wget https://example.com/arquivo.zip

# Download com nome customizado
$ wget -O meu_arquivo.zip https://example.com/arquivo.zip

# Download em background
$ wget -b https://example.com/arquivo_grande.zip

# Continuar download interrompido
$ wget -c https://example.com/arquivo_grande.zip

# Download recursivo (site inteiro)
$ wget -r https://example.com

# Download recursivo com limite de profundidade
$ wget -r -l 2 https://example.com

# Download apenas arquivos específicos
$ wget -r -A "*.pdf" https://example.com/documentos

# Rejeitar certos arquivos
$ wget -r -R "*.jpg,*.png" https://example.com

# Download com autenticação
$ wget --user=usuario --password=senha https://example.com/protegido

# Download via FTP
$ wget ftp://ftp.example.com/arquivo.txt

# Mirror de site
$ wget --mirror --convert-links --adjust-extension \
  --page-requisites --no-parent https://example.com/pagina

# Limitar velocidade
$ wget --limit-rate=200k https://example.com/arquivo.zip

# Tentativas
$ wget --tries=10 https://example.com/arquivo.zip

# Timeout
$ wget --timeout=30 https://example.com/arquivo.zip

# Spoof user-agent
$ wget --user-agent="Mozilla/5.0" https://example.com

# Download de lista de URLs
$ wget -i urls.txt

# Não sobrescrever arquivos existentes
$ wget -nc https://example.com/arquivo.zip
```

```
# Forçar sobrescrita
$ wget -N https://example.com/arquivo.zip

# Log de download
$ wget -o log.txt https://example.com/arquivo.zip

# Quiet mode
$ wget -q https://example.com/arquivo.zip
```

6.7 traceroute — Trace Route to Host

O **traceroute** mostra o caminho (saltos) que os pacotes fazem até chegar ao destino.

```
# Traceroute básico
$ traceroute google.com

# Traceroute com ICMP (ao invés de UDP)
$ traceroute -I google.com

# Traceroute com TCP
$ traceroute -T google.com

# Traceroute com porta específica
$ traceroute -p 80 google.com

# Limitar número de saltos
$ traceroute -m 15 google.com

# Timeout por sonda
$ traceroute -w 5 google.com

# Número de sondas por salto
$ traceroute -q 3 google.com

# Não resolver nomes (mais rápido)
$ traceroute -n google.com

# Traceroute IPv6
$ traceroute6 google.com

# Usar interface específica
$ traceroute -i eth0 google.com
```

Interpretando traceroute

Cada linha representa um salto (router). Os três tempos são RTT de três sondas. *** * *** significa timeout (firewall bloqueando ICMP/UDP, ou router não responde). Saltos altos de RTT indicam congestionamento ou rota subótima.

6.8 dig — DNS Lookup Utility

O **dig** é a ferramenta mais completa para consultas DNS.

```
# Consulta A record
$ dig google.com

# Consulta específica
$ dig google.com A
$ dig google.com AAAA
$ dig google.com MX
$ dig google.com NS
$ dig google.com TXT
$ dig google.com CNAME
$ dig google.com SOA

# Consulta reversa (IP para nome)
$ dig -x 8.8.8.8

# Consultar servidor DNS específico
$ dig @8.8.8.8 google.com

# Consulta curta (apenas resposta)
$ dig +short google.com

# Consulta com trace completo
$ dig +trace google.com

# Mostrar apenas seção de resposta
$ dig +noall +answer google.com

# Consulta com timeout
$ dig +time=5 google.com

# Consulta com tentativas
$ dig +retry=3 google.com

# Não usar search domains
$ dig +nostdns google.com

# Consulta ANY (todos os registros)
$ dig google.com ANY

# Transferência de zona (requer permissão)
$ dig @ns.example.com example.com AXFR

# Consulta DNSSEC
$ dig +dnssec google.com

# Verificar assinatura DNSSEC
$ dig +sigchase google.com

# Consulta com IPv6
$ dig -6 google.com
```

6.8.1 Interpretando saída do dig

```
$ dig +noall +answer google.com
google.com.      285      IN      A       142.250.80.46

# google.com.    → nome consultado
# 285            → TTL (segundos)
# IN            → classe (Internet)
```

```
# A      → tipo de registro
# 142.250... → valor
```

6.9 nslookup — Query Internet Name Servers

O `nslookup` é uma alternativa mais simples ao `dig`, embora considerado legado.

```
# Consulta básica
$ nslookup google.com

# Consulta tipo específico
$ nslookup -query=MX google.com

# Consultar servidor específico
$ nslookup google.com 8.8.8.8

# Modo interativo
$ nslookup
> server 8.8.8.8
> set type=MX
> google.com
> exit

# Consulta reversa
$ nslookup 8.8.8.8
```

🔧 Exercícios — Volume 6

1. Use `ip addr show` e identifique: seu IP local, máscara de rede, e MAC address.
2. Use `ip route show` para encontrar seu gateway padrão. Use `ip route get 8.8.8.8` e interprete a saída.
3. Execute `ping -c 10 google.com` e analise: TTL, RTT médio, packet loss.
4. Use `ss -tlnp` para listar todos os serviços escutando em portas TCP. Identifique 3 serviços.
5. Use `curl -I` para ver os headers HTTP de um site. Identifique: servidor, content-type, status code.
6. Use `curl` para fazer um POST com JSON para `httpbin.org/post` e verifique a resposta.
7. Use `wget --mirror` para fazer mirror de um site pequeno (use um site de teste).
8. Execute `traceroute google.com` e conte quantos saltos até o destino. Identifique algum salto com latência anômala.
9. Use `dig +trace` para ver a resolução completa de um domínio, desde os root servers.
10. Use `dig` para consultar registros MX, NS, e TXT de um domínio.
11. Use `ss` para encontrar todas as conexões estabelecidas do seu sistema. Identifique para onde elas vão.
12. Crie um script que use `curl` para verificar se um site está online (HTTP 200) e logue o resultado.



Volume 7 — Gerenciamento de Pacotes

Instalar, atualizar e remover software no Debian/Ubuntu com apt e dpkg.

7.1 apt — Advanced Package Tool

O **apt** é a interface de alto nível para gerenciamento de pacotes em sistemas Debian/Ubuntu. Ele lida com dependências, repositórios, e resolução de conflitos.

7.1.1 Atualizar Informações de Pacotes

```
# Atualizar lista de pacotes disponíveis
$ sudo apt update

# Atualizar com debug
$ sudo apt update -o Debug::Acquire::http=true

# Atualizar apenas um componente
$ sudo apt update -o Dir::Etc::sourcelist="sources.list.d/meu_repo.list"
```

7.1.2 Instalar Pacotes

```
# Instalar pacote
$ sudo apt install nginx

# Instalar múltiplos pacotes
$ sudo apt install nginx mysql-server php

# Instalar sem recomendações
$ sudo apt install --no-install-recommends nginx

# Instalar versão específica
$ sudo apt install nginx=1.18.0-0ubuntu1

# Simular instalação (não instala, apenas mostra o que faria)
$ sudo apt install --simulate nginx
$ sudo apt install -s nginx

# Baixar apenas (não instalar)
$ sudo apt install --download-only nginx
# Pacotes vão para /var/cache/apt/archives/

# Instalar de arquivo .deb
$ sudo apt install ./pacote.deb

# Instalar com confirmação automática
$ sudo apt install -y nginx

# Instalar com reinstalação completa
$ sudo apt install --reinstall nginx
```


7.1.3 Remover Pacotes

```
# Remover pacote (mantém configurações)
$ sudo apt remove nginx

# Remover completamente (incluindo configurações)
$ sudo apt purge nginx
$ sudo apt remove --purge nginx

# Remover pacotes órfãos (dependências não mais necessárias)
$ sudo apt autoremove

# Remover pacotes órfãos com confirmação
$ sudo apt autoremove -y

# Remover pacotes baixados do cache
$ sudo apt clean

# Remover pacotes antigos do cache
$ sudo apt autoclean
```

7.1.4 Atualizar Sistema

```
# Atualizar pacotes instalados (sem remover nada)
$ sudo apt upgrade

# Atualizar com remoção de pacotes obsoletos
$ sudo apt full-upgrade
# (antigamente: apt dist-upgrade)

# Atualização não-interativa
$ sudo apt upgrade -y

# Atualizar apenas pacotes de segurança
$ sudo apt upgrade -y -o APT::Get::Upgrade-Allow-New=false

# Simular atualização
$ sudo apt upgrade --simulate
```

7.1.5 Buscar e Informações

```
# Buscar pacote por nome/descrição
$ apt search nginx

# Buscar com regex
$ apt search ^nginx

# Informações detalhadas do pacote
$ apt show nginx

# Listar dependências
$ apt depends nginx

# Listar dependências reversas (quem depende deste)
$ apt rdepends nginx

# Listar arquivos de um pacote instalado
```

```
$ dpkg -L nginx

# Listar pacotes instalados
$ apt list --installed

# Verificar se pacote está instalado
$ apt list --installed | grep nginx

# Verificar versão instalada
$ apt list --installed nginx

# Política de versões (repositórios)
$ apt policy nginx

# Cache de pacotes
$ apt cache show nginx
```

7.1.6 Gerenciar Repositórios

```
# Listar repositórios configurados
$ cat /etc/apt/sources.list
$ ls /etc/apt/sources.list.d/

# Adicionar repositório (Debian/Ubuntu)
$ sudo add-apt-repository ppa:usuario/repo

# Adicionar repositório manualmente
$ echo "deb http://repo.example.com/ubuntu jammy main" | sudo tee /etc/apt/sources.list.d/meu_repo.list

# Adicionar chave GPG
$ wget -qO - https://repo.example.com/key.gpg | sudo apt-key add -
# ou (moderno):
$ sudo wget -qO /usr/share/keyrings/meu_repo.gpg https://repo.example.com/key.gpg

# Remover repositório
$ sudo add-apt-repository --remove ppa:usuario/repo
$ sudo rm /etc/apt/sources.list.d/meu_repo.list

# Editar sources.list com editor seguro
$ sudo apt edit-sources
```

7.2 dpkg — Debian Package Manager

O **dpkg** é a ferramenta de baixo nível para manipular pacotes .deb. Não resolve dependências automaticamente — use **apt** para isso.

7.2.1 Instalar e Remover

```
# Instalar pacote .deb
$ sudo dpkg -i pacote.deb

# Instalar com resolução de dependências (use apt)
$ sudo apt install ./pacote.deb

# Remover pacote
```

```
$ sudo dpkg -r pacote

# Remover completamente
$ sudo dpkg -P pacote

# Reconfigurar pacote
$ sudo dpkg-reconfigure pacote

# Reconfigurar todos os pacotes
$ sudo dpkg-reconfigure -a
```

7.2.2 Consultar Informações

```
# Listar pacotes instalados
$ dpkg -l
$ dpkg -l | grep nginx

# Informações de um pacote
$ dpkg -s nginx

# Listar arquivos de um pacote
$ dpkg -L nginx

# Descobrir qual pacote possui um arquivo
$ dpkg -S /usr/bin/nginx
$ dpkg -S /etc/nginx/nginx.conf

# Status de um pacote
$ dpkg -s nginx | grep Status

# Listar arquivos de configuração
$ dpkg -L nginx | grep /etc/

# Verificar integridade de pacotes
$ dpkg -V
$ dpkg -V nginx
```

7.2.3 Depuração e Reparo

```
# Verificar pacotes quebrados
$ dpkg -C

# Reconfigurar pacotes desconfigurados
$ sudo dpkg --configure -a

# Forçar reinstalação
$ sudo dpkg -i --force-all pacote.deb

# Ignorar conflitos
$ sudo dpkg -i --force-overwrite pacote.deb

# Remover forçado
$ sudo dpkg -r --force-depends pacote

# Limpar estado de pacote
$ sudo dpkg --clear-avail

# Atualizar banco de dados
```

```
$ sudo dpkg --update-avail
```

7.3 apt-cache — Query the APT Cache

```
# Buscar pacotes
$ apt-cache search nginx

# Informações detalhadas
$ apt-cache show nginx

# Dependências
$ apt-cache depends nginx

# Dependências reversas
$ apt-cache rdepends nginx

# Estatísticas do cache
$ apt-cache stats

# Política de versão
$ apt-cache policy nginx

# Listar pacotes disponíveis
$ apt-cache pkgnames
$ apt-cache pkgnames nginx
```

7.4 apt-get vs. apt

Comando	apt-get	apt
Atualizar lista	<code>apt-get update</code>	<code>apt update</code>
Instalar	<code>apt-get install</code>	<code>apt install</code>
Remover	<code>apt-get remove</code>	<code>apt remove</code>
Purge	<code>apt-get purge</code>	<code>apt purge</code>
Upgrade	<code>apt-get upgrade</code>	<code>apt upgrade</code>
Full-upgrade	<code>apt-get dist-upgrade</code>	<code>apt full-upgrade</code>
Autoremove	<code>apt-get autoremove</code>	<code>apt autoremove</code>
Clean	<code>apt-get clean</code>	<code>apt clean</code>
Buscar	<code>apt-cache search</code>	<code>apt search</code>
Mostrar	<code>apt-cache show</code>	<code>apt show</code>
Progresso	Sem barra	Com barra de progresso
Output	Mais técnico	Mais amigável

Recomendação

Use `apt` para uso interativo (mais amigável). Use `apt-get` em scripts (mais estável, output mais previsível). `apt` é um wrapper que chama `apt-get` e `apt-cache` por baixo.

🔧 Exercícios — Volume 7

1. Execute `apt update` e observe a saída. Quantos pacotes podem ser atualizados?
2. Use `apt search` para encontrar 3 pacotes relacionados a "editor de texto".
3. Use `apt show` para ver informações detalhadas de um pacote instalado. Identifique: versão, tamanho, dependências, descrição.
4. Use `dpkg -l` e conte quantos pacotes estão instalados no seu sistema.
5. Use `dpkg -S` para descobrir qual pacote fornece o comando `ls`.
6. Use `apt list --installed` e filtre apenas pacotes que começam com "lib".
7. Verifique `apt policy` de um pacote e interprete: qual versão está instalada? Qual está disponível? De qual repositório?
8. Execute `apt autoremove --simulate` e veja quais pacotes seriam removidos. Execute de verdade se fizer sentido.
9. Use `apt-cache depends` e `apt-cache rdepends` para um pacote e trace a árvore de dependências.
10. Crie um script que verifique se há atualizações pendentes e envie um alerta se houver.

Volume 8 — Bash Scripting

Automatize tarefas, crie ferramentas e domine a programação no shell.

8.1 Shebang e Estrutura de um Script

```
#!/bin/bash
#=====
# Nome: backup.sh
# Descrição: Faz backup de diretórios importantes
# Autor: Seu Nome
# Data: 2024-06-14
# Uso: ./backup.sh [diretório]
#=====

set -euo pipefail
# -e: sai se algum comando falhar
# -u: erro ao usar variável não definida
# -o pipefail: pipeline falha se qualquer comando falhar

# Variáveis
BACKUP_DIR="/backup"
DATA=$(date +%Y-%m-%d)
LOG="$BACKUP_DIR/backup_$(date +%Y-%m-%d).log"

# Funções
log() {
    echo "[$(date +%Y-%m-%d %H:%M:%S)] $1" | tee -a "$LOG"
}

# Código principal
log "Iniciando backup..."
# ...
log "Backup concluído!"
```

Shebangs Comuns

Shebang	Significado
<code>#!/bin/bash</code>	Bash (caminho absoluto)
<code>#!/usr/bin/env bash</code>	Bash (mais portátil)
<code>#!/bin/sh</code>	Shell POSIX (mais portátil ainda)
<code>#!/bin/bash -x</code>	Modo debug (mostra cada comando)
<code>#!/bin/bash -e</code>	Exit on error

8.2 Variaveis

8.2.1 Declaracao e Uso

```
#!/bin/bash

# Declaracao (SEM espacos ao redor de =)
NOME="Joao"
IDADE=25
PI=3.14159

# Uso (com $)
echo "Ola, $NOME!"
echo "Voce tem ${IDADE} anos"

# Variavel nao definida (com -u, gera erro)
# echo "$NAO_EXISTE" # ERRO!

# Valor padrao
echo "${NAO_EXISTE:-padrao}" # imprime "padrao"
echo "${NAO_EXISTE:=padrao}" # define e imprime "padrao"
echo "${EXISTE:?\"nao definida\"}" # erro se nao definida

# Remover prefixo/sufixo
ARQUIVO="documento.txt"
echo "${ARQUIVO%.txt}" # documento

echo "${ARQUIVO#doc}" # umento.txt

# Substituicao
TEXT0="hello world hello"
echo "${TEXT0/hello/HI}" # HI world hello (primeira)
echo "${TEXT0//hello/HI}" # HI world HI (todas)

# Maiusculas/minusculas
echo "${TEXT0^^}" # HELLO WORLD HELLO
echo "${TEXT0,,}" # hello world hello

# Comprimento
echo "${#TEXT0}" # 17

# Substring
TEXT0="abcdef"
echo "${TEXT0:2}" # cdef (a partir do indice 2)
echo "${TEXT0:2:3}" # cde (3 caracteres a partir do 2)
```

8.2.2 Variaveis de Ambiente

```
# Ver todas
$ env
$ printenv

# Ver uma especifica
$ echo $HOME
$ echo $PATH
$ echo $USER
$ echo $SHELL
```

```

# Definir temporariamente
$ VAR=valor comando

# Exportar para subprocessos
$ export MINHA_VAR="valor"

# Remover
$ unset MINHA_VAR

# Variaveis importantes do bash
$ echo $0      # nome do script
$ echo $1 $2 $3 # argumentos posicionais
$ echo @$      # todos os argumentos (lista)
$ echo $*      # todos os argumentos (string)
$ echo $#      # numero de argumentos
$ echo $?      # codigo de saida do ultimo comando
$ echo $$      # PID do shell atual
$ echo $!      # PID do ultimo job em background
$ echo $_      # ultimo argumento do ultimo comando

```

8.2.3 Arrays

```

#!/bin/bash

# Declarar array
FRUTAS=("maca" "banana" "laranja")

# Acessar elementos
echo "${FRUTAS[0]}" # maca
echo "${FRUTAS[1]}" # banana

# Todos os elementos
echo "${FRUTAS[@]}" # maca banana laranja
echo "${FRUTAS[*]}" # maca banana laranja

# Numero de elementos
echo "${#FRUTAS[@]}" # 3

# Adicionar elemento
FRUTAS+=("uva")

# Iterar
for fruta in "${FRUTAS[@]}"; do
    echo "Fruta: $fruta"
done

# Iterar com indice
for i in "${!FRUTAS[@]}"; do
    echo "$i: ${FRUTAS[$i]}"
done

# Remover elemento
unset FRUTAS[1]

# Array associativo (bash 4+)
declare -A USUARIOS
USUARIOS[joao]=25
USUARIOS[maria]=30
echo "${USUARIOS[joao]}" # 25

```



```
# Todas as chaves
echo "${!USUARIOS[@]}"      # joao maria
```

8.3 Condicionais — if, case

8.3.1 if

```
#!/bin/bash

# Basico
if [ "$IDADE" -ge 18 ]; then
    echo "Maior de idade"
fi

# if-else
if [ "$IDADE" -ge 18 ]; then
    echo "Maior de idade"
else
    echo "Menor de idade"
fi

# if-elif-else
if [ "$NOTA" -ge 90 ]; then
    echo "A"
elif [ "$NOTA" -ge 80 ]; then
    echo "B"
elif [ "$NOTA" -ge 70 ]; then
    echo "C"
else
    echo "D"
fi

# Testes com && e ||
[ -f "$ARQUIVO" ] && echo "Existe" || echo "Nao existe"

# Teste inverso
if ! [ -f "$ARQUIVO" ]; then
    echo "Arquivo nao existe"
fi
```

8.3.2 Operadores de Teste

Operador	Significado
Strings	
<code>-z "\$VAR"</code>	String vazia
<code>-n "\$VAR"</code>	String nao vazia
<code>"\$A" = "\$B"</code>	Iguais
<code>"\$A" != "\$B"</code>	Diferentes
Numeros	

<code>-eq</code>	Igual (equal)
<code>-ne</code>	Diferente (not equal)
<code>-lt</code>	Menor que (less than)
<code>-le</code>	Menor ou igual
<code>-gt</code>	Maior que (greater than)
<code>-ge</code>	Maior ou igual
Arquivos	
<code>-e</code>	Existe
<code>-f</code>	Arquivo regular
<code>-d</code>	Diretorio
<code>-L</code>	Link simbolico
<code>-r</code>	Legivel
<code>-w</code>	Gravavel
<code>-x</code>	Executavel
<code>-s</code>	Nao vazio (tem conteudo)
<code>-N</code>	Mais novo que
<code>-O</code>	Dono eh o usuario atual
<code>-G</code>	Grupo eh o grupo atual

8.3.3 case

```
#!/bin/bash

case "$1" in
    start)
        echo "Iniciando servico..."
        ;;
    stop)
        echo "Parando servico..."
        ;;
    restart)
        echo "Reiniciando servico..."
        ;;
    status)
        echo "Status do servico..."
        ;;
    *)
        echo "Uso: $0 {start|stop|restart|status}"
        exit 1

```

```

;;
esac

# case com padroes
case "$EXT" in
    *.txt|*.md)
        echo "Texto"
        ;;
    *.jpg|*.png|*.gif)
        echo "Imagem"
        ;;
    *.mp3|*.wav)
        echo "Audio"
        ;;
    *)
        echo "Desconhecido"
        ;;
esac

```

8.4 Loops — for, while, until

8.4.1 for

```

#!/bin/bash

# Iterar sobre lista
for i in 1 2 3 4 5; do
    echo "Numero: $i"
done

# Range (bash 4+)
for i in {1..10}; do
    echo "$i"
done

# Range com passo
for i in {1..10..2}; do
    echo "$i" # 1, 3, 5, 7, 9
done

# Iterar sobre array
for fruta in "${FRUTAS[@]}"; do
    echo "$fruta"
done

# Iterar sobre arquivos
for arquivo in *.txt; do
    echo "Processando: $arquivo"
done

# C-style for
for ((i=0; i<10; i++)); do
    echo "$i"
done

# Ler linhas de arquivo
while IFS= read -r linha; do
    echo "$linha"
done

```

```
done < arquivo.txt

# Iterar sobre saida de comando
for usuario in $(cut -d: -f1 /etc/passwd); do
    echo "Usuario: $usuario"
done

# Iterar com find (seguro para nomes com espacos)
while IFS= read -r -d '' arquivo; do
    echo "Arquivo: $arquivo"
done < <(find . -name "*.txt" -print0)
```

8.4.2 while

```
#!/bin/bash

# Loop basico
CONTADOR=0
while [ "$CONTADOR" -lt 10 ]; do
    echo "$CONTADOR"
    ((CONTADOR++))
done

# Ler arquivo linha por linha
while IFS= read -r linha; do
    echo "Linha: $linha"
done < arquivo.txt

# Loop infinito
while true; do
    echo "Executando..."
    sleep 5
done

# Loop com condicao de arquivo
while [ -f "/tmp/lock" ]; do
    echo "Aguardando..."
    sleep 1
done

# Loop com read (input do usuario)
while true; do
    read -p "Digite 'sair' para encerrar: " resposta
    [ "$resposta" = "sair" ] && break
done
```

8.4.3 until

```
#!/bin/bash

# Executa ATE a condicao ser verdadeira
CONTADOR=0
until [ "$CONTADOR" -ge 10 ]; do
    echo "$CONTADOR"
    ((CONTADOR++))
done

# Esperar arquivo aparecer
```

```
until [ -f "/tmp/pronto" ]; do
    echo "Aguardando arquivo..."
    sleep 1
done
```

8.4.4 Controle de Loop

```
#!/bin/bash

# break: sai do loop
for i in {1..10}; do
    [ "$i" -eq 5 ] && break
    echo "$i"
done
# Sai no 5, imprime 1-4

# continue: pula para proxima iteracao
for i in {1..10}; do
    [ "$i" -eq 5 ] && continue
    echo "$i"
done
# Pula o 5, imprime 1-4, 6-10

# break N: sai de N niveis de loop
for i in {1..3}; do
    for j in {1..3}; do
        [ "$j" -eq 2 ] && break 2
        echo "$i-$j"
    done
done
# Sai dos dois loops
```

8.5 Funcoes

```
#!/bin/bash

# Declaracao basica
minha_funcao() {
    echo "Ola da funcao!"
}

# Ou
function minha_funcao {
    echo "Ola da funcao!"
}

# Chamada
minha_funcao

# Com parametros
soma() {
    local a=$1
    local b=$2
    echo $((a + b))
}

RESULTADO=$(soma 5 3)
```

```

echo "Resultado: $RESULTADO"

# Retorno (codigo de saida)
verificar_arquivo() {
    if [ -f "$1" ]; then
        return 0 # sucesso
    else
        return 1 # falha
    fi
}

if verificar_arquivo "/etc/passwd"; then
    echo "Arquivo existe"
else
    echo "Arquivo nao existe"
fi

# Variaveis locais
funcao() {
    local VAR="local"
    echo "$VAR"
}
funcao
echo "$VAR" # vazio (nao definido no escopo global)

# Funcao recursiva
fatorial() {
    local n=$1
    if [ "$n" -le 1 ]; then
        echo 1
    else
        local prev=$(fatorial $((n - 1)))
        echo $((n * prev))
    fi
}

echo "5! = $(fatorial 5)"

# Passar array para funcao
processar_array() {
    local arr=("$@")
    for item in "${arr[@]}; do
        echo "Item: $item"
    done
}

processar_array "${FRUTAS[@]}"

```

8.6 Parametros e Argumentos

```

#!/bin/bash

# Nome do script
SCRIPT="$0"

# Argumentos posicionais
PRIMEIRO="$1"
SEGUNDO="$2"
TERCEIRO="$3"

```

```

# Todos os argumentos
TODOS="$@"
TODOS_STRING="$*"

# Numero de argumentos
NUMERO="$#"

# Ultimo argumento
ULTIMO="${!#}"

# Shift (remove primeiro argumento)
shift
echo "Agora \ $1 eh: $1"

# Shift N
shift 2

# getopt (parse de opcoes)
while getopt "a:b:vh" opt; do
    case $opt in
        a)
            ARG_A="$OPTARG"
            ;;
        b)
            ARG_B="$OPTARG"
            ;;
        v)
            VERBOSE=1
            ;;
        h)
            echo "Uso: $0 -a valor -b valor [-v] [-h]"
            exit 0
            ;;
        \?)
            echo "Opcao invalida: -$OPTARG" >&2
            exit 1
            ;;
    esac
done
shift $((OPTIND - 1))

# Uso: ./script.sh -a valor1 -b valor2 -v arquivo.txt

# Argumentos restantes apos getopt
echo "Arquivos: $@"

```

8.7 Variaveis Especiais

Variavel	Significado
<code>\$0</code>	Nome do script/comando
<code>\$1, \$2, ... \$9</code>	Argumentos posicionais
<code>\${10}, \${11}, ...</code>	Argumentos apos o 9o
<code>\$@</code>	Todos os argumentos (lista)

<code>\$*</code>	Todos os argumentos (string)
<code>\$#</code>	Numero de argumentos
<code>\$?</code>	Codigo de saida do ultimo comando
<code>\$\$</code>	PID do shell atual
<code>\$_</code>	PID do ultimo job em background
<code>\$_</code>	Ultimo argumento do ultimo comando
<code>\$-</code>	Flags do shell atuais

8.7.1 Codigos de Saida

Codigo	Significado
<code>0</code>	Sucesso
<code>1</code>	Erro generico
<code>2</code>	Uso incorreto do comando
<code>126</code>	Comando nao executavel
<code>127</code>	Comando nao encontrado
<code>128+N</code>	Morto pelo sinal N
<code>130</code>	Morto por Ctrl+C (SIGINT = 2, 128+2=130)
<code>137</code>	Morto por SIGKILL (9, 128+9=137)
<code>255</code>	Erro de saida

🔧 Exercicios — Volume 8

1. Crie um script com shebang, comentarios de cabecalho, e `set -euo pipefail`.
2. Crie um script que receba um nome como argumento e diga "Ola, [nome]!". Use valor padrao se nenhum argumento for passado.
3. Crie um script que verifique se um arquivo existe, se eh legivel, e se tem conteudo. Use operadores de teste.
4. Crie um script com `case` que simule um menu: 1=backup, 2=restaurar, 3=status, 4=sair.
5. Crie um script com `for` que processe todos os arquivos `.txt` de um diretorio e conte as linhas de cada um.
6. Crie um script com `while` que leia `/etc/passwd` e imprima apenas usuarios com UID ≥ 1000 .
7. Crie uma funcao que receba dois numeros e retorne a soma. Teste com diferentes valores.

8. Crie um script que use `getopts` para aceitar `-f arquivo`, `-v` (verbose), e `-h` (ajuda).
9. Crie um array com 5 linguagens de programacao e itere sobre ele, imprimindo cada uma em maiusculas.
10. Crie um script que monitore um diretorio: se um arquivo novo aparecer, imprima uma mensagem (use loop infinito).

Volume 9 — Ferramentas para Desenvolvimento

Compilar, depurar, versionar e transferir código como um profissional.

9.1 gcc — GNU Compiler Collection

O `gcc` é o compilador C padrão no Linux. Compila código C em executáveis, bibliotecas e objetos.

9.1.1 Compilacao Basica

```
# Compilar um arquivo C
$ gcc programa.c -o programa

# Compilar e executar
$ gcc programa.c -o programa && ./programa

# Compilar com warnings
$ gcc -Wall programa.c -o programa

# Compilar com warnings extras
$ gcc -Wall -Wextra programa.c -o programa

# Compilar com todas as warnings
$ gcc -Wall -Wextra -Wpedantic programa.c -o programa

# Compilar com debug info
$ gcc -g programa.c -o programa

# Compilar com otimizacao
$ gcc -O2 programa.c -o programa

# Compilar com otimizacao maxima
$ gcc -O3 programa.c -o programa

# Compilar sem otimizacao (para debug)
$ gcc -O0 -g programa.c -o programa

# Compilar com sanitizers (deteccao de erros em runtime)
$ gcc -fsanitize=address programa.c -o programa
$ gcc -fsanitize=undefined programa.c -o programa
```

9.1.2 Flags Importantes do gcc

Flag	Significado
<code>-o arquivo</code>	Nome do arquivo de saída
<code>-c</code>	Compilar apenas (não linkar)

<code>-S</code>	Gerar assembly
<code>-E</code>	Preprocessar apenas
<code>-g</code>	Incluir informacoes de debug
<code>-O0</code>	Sem otimizacao
<code>-O1</code>	Otimizacao basica
<code>-O2</code>	Otimizacao padrao
<code>-O3</code>	Otimizacao agressiva
<code>-Os</code>	Otimizar para tamanho
<code>-Wall</code>	Warnings comuns
<code>-Wextra</code>	Warnings extras
<code>-Werror</code>	Tratar warnings como erros
<code>-std=c11</code>	Padrao C11
<code>-std=c17</code>	Padrao C17
<code>-I dir</code>	Diretorio de includes
<code>-L dir</code>	Diretorio de bibliotecas
<code>-l lib</code>	Linkar com biblioteca
<code>-static</code>	Linkagem estatica
<code>-shared</code>	Criar biblioteca compartilhada
<code>-fPIC</code>	Position Independent Code
<code>-D MACRO</code>	Definir macro
<code>-U MACRO</code>	Remover macro
<code>-M</code>	Gerar dependencias
<code>-MM</code>	Gerar dependencias (sem sistema)

9.1.3 Compilacao em Multiplos Passos

```
# 1. Preprocessar
$ gcc -E programa.c -o programa.i

# 2. Compilar para assembly
$ gcc -S programa.i -o programa.s

# 3. Montar (assembler)
$ gcc -c programa.s -o programa.o
```

```
# 4. Linkar
$ gcc programa.o -o programa

# Ou tudo de uma vez:
$ gcc programa.c -o programa

# Compilar multiplos arquivos
$ gcc -c main.c -o main.o
$ gcc -c utils.c -o utils.o
$ gcc main.o utils.o -o programa
```

Codigo C (.c) | v Preprocessador (cpp) → Codigo preprocessado (.i) | v Compilador (cc1) → Assembly (.s) | v Assembler (as) → Objeto (.o) | v Linker (ld) → Executavel

9.2 make — Build Automation

O **make** automatiza a compilacao, gerenciando dependencias entre arquivos.

9.2.1 Makefile Basico

```
# Makefile

CC = gcc
CFLAGS = -Wall -Wextra -g -O2
TARGET = programa
SRCS = main.c utils.c
OBJS = $(SRCS:.c=.o)

.PHONY: all clean

all: $(TARGET)

$(TARGET): $(OBJS)
    $(CC) $(OBJS) -o $(TARGET)

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

clean:
    rm -f $(OBJS) $(TARGET)

# Uso:
# make          → compila tudo
# make clean    → remove objetos e executavel
# make main.o   → compila apenas main.c
```

9.2.2 Variaveis Automaticas do make

Variavel	Significado
<code>\$@</code>	Nome do alvo (target)
<code>\$<</code>	Primeira dependencia

`$^`

Todas as dependencias

`$?`

Dependências mais novas que o alvo

`$*`

Stem (parte do padrão %)

9.2.3 Makefile Avancado

```
# Makefile avancado

CC = gcc
CFLAGS = -Wall -Wextra -g -O2
LDFLAGS = -lm -lpthread
TARGET = programa
SRCDIR = src
OBJDIR = obj
BINDIR = bin

SRCS = $(wildcard $(SRCDIR)/*.c)
OBJS = $(patsubst $(SRCDIR)/%.c,$(OBJDIR)/%.o,$(SRCS))

.PHONY: all clean dirs debug release

all: dirs $(BINDIR)/$(TARGET)

dirs:
    mkdir -p $(OBJDIR) $(BINDIR)

$(BINDIR)/$(TARGET): $(OBJS)
    $(CC) $(OBJS) -o $@ $(LDFLAGS)

$(OBJDIR)/%.o: $(SRCDIR)/%.c
    $(CC) $(CFLAGS) -c $< -o $@

clean:
    rm -rf $(OBJDIR) $(BINDIR)

debug: CFLAGS = -Wall -Wextra -g -O0 -DDEBUG
debug: all

release: CFLAGS = -Wall -Wextra -O3 -DNDEBUG
release: all

# Uso:
# make      → build padrao
# make debug → build com debug
# make release → build otimizada
# make clean → limpa tudo
```

9.3 gdb — GNU Debugger

O `gdb` é o depurador para C/C++. Permite inspecionar variáveis, breakpoints, stack trace, e muito mais.

9.3.1 Iniciar o gdb

```
# Compilar com debug info
```

```

$ gcc -g programa.c -o programa

# Iniciar gdb
$ gdb ./programa

# Iniciar e passar argumentos
$ gdb --args ./programa arg1 arg2

# Analisar core dump
$ gdb ./programa core

# Anexar a processo em execucao
$ gdb -p 1234

```

9.3.2 Comandos do gdb

Comando	Abrev	Descricao
<code>run</code>	<code>r</code>	Executar programa
<code>break</code>	<code>b</code>	Definir breakpoint
<code>continue</code>	<code>c</code>	Continuar execucao
<code>next</code>	<code>n</code>	Proxima linha (nao entra em funcoes)
<code>step</code>	<code>s</code>	Proxima linha (entra em funcoes)
<code>finish</code>		Executa ate sair da funcao atual
<code>print</code>	<code>p</code>	Imprimir valor de expressao
<code>display</code>		Mostrar valor a cada parada
<code>backtrace</code>	<code>bt</code>	Stack trace
<code>info locals</code>		Variaveis locais
<code>info args</code>		Argumentos da funcao
<code>list</code>	<code>l</code>	Mostrar codigo fonte
<code>set var</code>		Alterar valor de variavel
<code>watch</code>		Breakpoint em mudanca de valor
<code>delete</code>	<code>d</code>	Remover breakpoints
<code>quit</code>	<code>q</code>	Sair

9.3.3 Exemplo de Sessao gdb

```

$ gdb ./programa
(gdb) break main
(gdb) run

```

```
(gdb) next
(gdb) print variavel
(gdb) continue
(gdb) backtrace
(gdb) info locals
(gdb) quit
```

9.3.4 gdb Avancado

```
# Breakpoint condicional
(gdb) break main.c:25 if x > 10

# Breakpoint em funcao
(gdb) break minha_funcao

# Breakpoint com comando automatico
(gdb) break main.c:30
(gdb) commands
> silent
> print variavel
> continue
> end

# Watchpoint (parar quando valor muda)
(gdb) watch variavel
(gdb) watch *(int*)0x12345678

# Reverse debugging
(gdb) record
(gdb) reverse-next
(gdb) reverse-step
(gdb) reverse-continue

# Core dump analysis
$ ulimit -c unlimited
$ ./programa # gera core dump em crash
$ gdb ./programa core
(gdb) bt full
(gdb) info registers
(gdb) x/10i $pc

# TUI mode (interface grafica no terminal)
(gdb) layout src
(gdb) layout asm
(gdb) layout regs
(gdb) focus cmd

# Pretty printing
(gdb) set print pretty on
(gdb) p array

# Scripts gdb
$ gdb -x script.gdb ./programa
```

9.4 git — Version Control

O **git** é o sistema de controle de versao mais usado no mundo. Essencial para qualquer desenvolvedor.

9.4.1 Comandos Essenciais

```
# Inicializar repositório
$ git init

# Clonar repositório
$ git clone https://github.com/usuario/repo.git
$ git clone https://github.com/usuario/repo.git meu_repo

# Ver status
$ git status

# Adicionar arquivo ao stage
$ git add arquivo.txt
$ git add .                # todos
$ git add -p               # interativo (patch)

# Commit
$ git commit -m "Mensagem do commit"
$ git commit -am "Mensagem"    # add + commit (arquivos rastreados)

# Ver log
$ git log
$ git log --oneline
$ git log --graph --oneline --all
$ git log -p                 # com diff
$ git log --stat              # com estatísticas

# Ver diff
$ git diff                   # working vs staged
$ git diff --staged           # staged vs last commit
$ git diff HEAD~1             # diff com commit anterior
```

9.4.2 Branches

```
# Listar branches
$ git branch
$ git branch -a              # todas (incluindo remotas)
$ git branch -v              # com último commit

# Criar branch
$ git branch nova-feature

# Mudar de branch
$ git checkout nova-feature
$ git switch nova-feature    # novo comando (git 2.23+)

# Criar e mudar
$ git checkout -b nova-feature
$ git switch -c nova-feature  # novo comando

# Merge
$ git checkout main
$ git merge nova-feature

# Rebase
$ git checkout nova-feature
$ git rebase main
```



```
# Deletar branch
$ git branch -d nova-feature    # merged
$ git branch -D nova-feature    # force delete

# Renomear branch
$ git branch -m novo-nome
```

9.4.3 Remoto

```
# Ver remotos
$ git remote -v

# Adicionar remoto
$ git remote add origin https://github.com/usuario/repo.git

# Fetch (baixa sem merge)
$ git fetch origin

# Pull (fetch + merge)
$ git pull origin main

# Push
$ git push origin main
$ git push -u origin main    # upstream

# Push de nova branch
$ git push -u origin nova-feature

# Deletar branch remota
$ git push origin --delete nova-feature

# Tags
$ git tag v1.0.0
$ git push origin v1.0.0
$ git push origin --tags
```

9.4.4 Desfazer e Corrigir

```
# Desfazer alteracoes nao staged
$ git checkout -- arquivo.txt
$ git restore arquivo.txt    # novo comando

# Unstage
$ git reset HEAD arquivo.txt
$ git restore --staged arquivo.txt    # novo comando

# Amend (alterar ultimo commit)
$ git commit --amend -m "Nova mensagem"
$ git commit --amend --no-edit    # sem alterar mensagem

# Reset
$ git reset --soft HEAD~1    # desfaz commit, mantem staged
$ git reset --mixed HEAD~1    # desfaz commit, mantem working
$ git reset --hard HEAD~1    # desfaz TUDO (CUIDADO!)

# Stash
$ git stash
$ git stash push -m "mensagem"
```

```
$ git stash list
$ git stash pop           # aplica e remove
$ git stash apply         # aplica sem remover
$ git stash drop          # remove
$ git stash clear         # remove todos

# Cherry-pick
$ git cherry-pick abc1234

# Revert (cria commit que desfaz)
$ git revert abc1234
```

9.4.5 git Avancado

```
# Bisect (encontrar commit que introduziu bug)
$ git bisect start
$ git bisect bad          # commit atual tem bug
$ git bisect good v1.0    # versao boa
# git testa commits intermediarios
$ git bisect good         # ou bad
$ git bisect reset       # termina

# Reflog (historico de todas as operacoes)
$ git reflog

# Blame (quem escreveu cada linha)
$ git blame arquivo.c

# Log com filtro
$ git log --author="Joao"
$ git log --since="2 weeks ago"
$ git log --grep="bugfix"

# Diff entre branches
$ git diff main..feature

# Submodules
$ git submodule add https://github.com/usuario/lib.git
$ git submodule update --init --recursive

# Worktree (multiplos working directories)
$ git worktree add ../repo-fix fix-branch

# Hooks
$ ls .git/hooks/
# pre-commit, post-commit, pre-push, etc.
```

9.5 ssh — Secure Shell

O `ssh` permite acesso remoto seguro a servidores Linux.

9.5.1 Conexao Basica

```
# Conectar a servidor remoto
$ ssh usuario@servidor.com
```

```
# Conectar com porta especifica
$ ssh -p 2222 usuario@servidor.com

# Conectar com chave privada
$ ssh -i ~/.ssh/minha_chave usuario@servidor.com

# Executar comando remoto
$ ssh usuario@servidor.com "ls -la"

# Executar comando remoto com stdin local
$ cat arquivo.txt | ssh usuario@servidor.com "cat > /remoto/arquivo.txt"
```

9.5.2 Gerenciamento de Chaves

```
# Gerar par de chaves
$ ssh-keygen -t ed25519 -C "email@example.com"
$ ssh-keygen -t rsa -b 4096 -C "email@example.com"

# Copiar chave publica para servidor
$ ssh-copy-id usuario@servidor.com

# Ou manualmente:
$ cat ~/.ssh/id_ed25519.pub | ssh usuario@servidor.com "mkdir -p ~/.ssh && cat >> ~/.ssh/authorized_keys"

# Iniciar agente SSH
$ eval "$(ssh-agent -s)"
$ ssh-add ~/.ssh/id_ed25519

# Listar chaves no agente
$ ssh-add -l

# Remover chave do agente
$ ssh-add -d ~/.ssh/id_ed25519

# Remover todas as chaves
$ ssh-add -D
```

9.5.3 Configuracao SSH

```
# ~/.ssh/config

Host meu-servidor
  HostName servidor.com
  User usuario
  Port 2222
  IdentityFile ~/.ssh/minha_chave
  ServerAliveInterval 60
  ServerAliveCountMax 3

Host github
  HostName github.com
  User git
  IdentityFile ~/.ssh/github_key

# Uso:
$ ssh meu-servidor
$ git clone github:usuario/repo.git
```

9.5.4 Tunelamento (Port Forwarding)

```
# Local forward: porta local -> porta remota
$ ssh -L 8080:localhost:80 usuario@servidor.com
# Acesse localhost:8080 para chegar ao servidor:80

# Remote forward: porta remota -> porta local
$ ssh -R 9090:localhost:3000 usuario@servidor.com
# No servidor, acesse localhost:9090 para chegar ao seu :3000

# Dynamic forward (SOCKS proxy)
$ ssh -D 1080 usuario@servidor.com
# Configure navegador para usar SOCKS5 localhost:1080

# Background (nao trava terminal)
$ ssh -f -N -L 8080:localhost:80 usuario@servidor.com

# Gateway ports (permitir acesso externo ao tunnel)
$ ssh -L 0.0.0.0:8080:localhost:80 usuario@servidor.com
```

9.6 scp — Secure Copy

O **scp** copia arquivos entre hosts via SSH.

```
# Copiar local para remoto
$ scp arquivo.txt usuario@servidor:/home/usuario/

# Copiar remoto para local
$ scp usuario@servidor:/home/usuario/arquivo.txt ./

# Copiar diretorio recursivamente
$ scp -r diretorio/ usuario@servidor:/home/usuario/

# Copiar com porta especifica
$ scp -P 2222 arquivo.txt usuario@servidor:/home/usuario/

# Copiar com chave especifica
$ scp -i ~/.ssh/minha_chave arquivo.txt usuario@servidor:/home/usuario/

# Copiar preservando atributos
$ scp -p arquivo.txt usuario@servidor:/home/usuario/

# Copiar com compressao
$ scp -C arquivo.txt usuario@servidor:/home/usuario/

# Copiar com limite de banda (KB/s)
$ scp -l 1000 arquivo.txt usuario@servidor:/home/usuario/

# Copiar multiplos arquivos
$ scp arquivo1.txt arquivo2.txt usuario@servidor:/home/usuario/

# Copiar com verbose
$ scp -v arquivo.txt usuario@servidor:/home/usuario/
```

Preferencia moderna: rsync

Para transferencias frequentes ou grandes, `rsync` eh geralmente preferido ao `scp` porque resume transferencias interrompidas, sincroniza apenas diferencas, e eh mais eficiente:

```
$ rsync -avz --progress arquivo.txt usuario@servidor:/home/usuario/  
$ rsync -avz --delete diretorio/ usuario@servidor:/home/usuario/diretorio/
```

🔧 Exercícios — Volume 9

1. Escreva um programa C simples, compile com `gcc -Wall -Wextra -g`, e execute. Depois compile com `-O3` e compare o tamanho do executavel.
2. Crie um Makefile para um projeto com `main.c` e `utils.c`. Teste `make`, `make clean`, e `make debug`.
3. Escreva um programa C com um bug (ex: acesso a array fora dos limites). Compile com `-g` e depure com `gdb`. Encontre o bug usando `break`, `run`, e `backtrace`.
4. Inicialize um repositorio git, faca 3 commits com mensagens descritivas, e use `git log --oneline -graph` para visualizar.
5. Crie uma branch, faca alteracoes, faca merge de volta para main, e depois delete a branch.
6. Use `git diff` para ver as diferencas entre working directory e staged. Use `git diff --staged` para ver staged vs last commit.
7. Gere um par de chaves SSH com `ssh-keygen` e configure o `~/.ssh/config` para um servidor ficticio.
8. Use `scp` para transferir um arquivo para um servidor remoto (ou simule com localhost).
9. Crie um tunel SSH local (`ssh -L`) e acesse um servico remoto atraves de uma porta local.
10. Use `git stash` para guardar alteracoes nao commitadas, faca outra tarefa, e depois recupere com `git stash pop`.